Use case reference

CROSSFADE, INFO 579 Project 2. 15 use cases from the Project 1 requirements analysis (RA). For each: the tables and columns it needs, which Project 2 file answers it, and, for each query in single_table.sql and multiple_table.sql, its input and expected output and every test case. Expected outputs are computed in Python from the Project 1 sample data (oracle.py, oracle_multiple.py) and were matched by MySQL in the last test runs: single_table.sql 2026-11-08 20:01:18 (71 passed, 0 failed); multiple_table.sql 2026-11-08 20:01:18 (80 passed, 0 failed). Day counts use the fixed report date 2026-09-30.

Summary

UC	Use case	Tables it needs	Answered by
UC-01	Pay each artist a per-stream royalty	artist, track, play	single_table.sql: S1; multiple_table.sql: M2
UC-02	Route payments by country; pay only verified artists	artist	single_table.sql: S1
UC-03	Surface new uploads as readily as hits	track	single_table.sql: S2
UC-04	Track the upload pipeline; reject duplicate uploads	track, track_score	single_table.sql: S7
UC-05	Show the genre legend while listeners compose a blend	genre	single_table.sql: S3
UC-06	Bill subscribers by plan tier from their signup date	listener	single_table.sql: S4
UC-07	Share stations under the creator's display name	listener, station, play	multiple_table.sql: M3
UC-08	Weight each model version by its measured accuracy	scoring_model, station	single_table.sql: S5, S6
UC-09	Find scoring results that predate a model redeployment	track_score, scoring_model, track	multiple_table.sql: M6
UC-10	Define stations by ambiguity window; measure how long they stay in use	station, station_blend	single_table.sql: S6; multiple_table.sql: M5
UC-11	Place each recording between genres	track_score, genre_probability, genre, track	multiple_table.sql: M4, M7
UC-12	Fill a station with recordings that match its blend and window	station, station_blend, track_score, genre_probability, track, genre	multiple_table.sql: M5
UC-13	Learn listener taste from completions, saves and skips	play, track, listener	multiple_table.sql: M1
UC-14	Keep an exact royalty ledger	play, track, artist	multiple_table.sql: M2
UC-15	Reject a duplicate upload	track, artist	single_table.sql: E1

UC-01 Pay each artist a per-stream royalty

Question: What is each artist paid per qualifying stream?

Amounts actually owed need play, track and artist together: [M2] in multiple_table.sql (LEFT OUTER JOIN, so artists owed nothing still appear). The royalty of one play (rule I7) is play_royalty, [F1] in routines.sql, which record_play [P1] stores on every play.

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`artist`	`artist_id, stage_name, payout_rate`	8 rows	alone: partial
`track`	`track_id, artist_id`	15 rows	with a join
`play`	`track_id, seconds_played, royalty_amount`	31 rows	with a join

S1 | crossfade.artist | single_table.sql

BUSINESS NEED

CROSSFADE pays every artist a per-stream royalty (RA: Overview; Artist). Two artist fields bear on whether a payment can go out: Verified, "whether the service has confirmed the artist's identity and their right to licence the material they upload", and Country, used "to determine which royalty payment channels and tax treatments apply" (RA: Artist).

QUESTION

Is each artist ready for this royalty run, and if not, which check is outstanding?

INPUT

The Project 1 sample data in `crossfade.artist` (8 rows), read through these columns: `artist_id, country, joined_date, payout_rate, stage_name, verified`. Brief requirements met: (a) (c) (e).

```
SELECT artist_id,
       stage_name,
       ROUND(payout_rate * 1000, 2) AS usd_per_1000_streams,
       DATEDIFF('2026-09-30', joined_date) AS days_under_licence,
       IF(verified = 1, 'confirmed', 'HOLD: not confirmed') AS licence_check,
       IF(country IS NULL, 'HOLD: no country', country) AS payment_country,
       CASE
           WHEN verified = 1 AND country IS NOT NULL THEN 'ready'
           ELSE 'held'
       END AS run_status
FROM crossfade.artist
ORDER BY run_status DESC,
         payout_rate DESC,
         artist_id;
```

EXPECTED OUTPUT: 8 row(s), in this order

artist_id	stage_name	usd_per_1000_streams	days_under_licence	licence_check	payment_country	run_status
1042	Velvet Static	4.00	200	confirmed	US	ready
1044	Low Orbit Choir	3.80	239	confirmed	CA	ready

artist_id	stage_name	usd_per_1000_streams	days_under_licence	licence_check	payment_country	run_status
1046	Ochre Parade	3.60	172	confirmed	MX	ready
1043	Mara Quell	3.50	253	confirmed	GB	ready
1047	Juno Fleet	3.00	151	confirmed	DE	ready
1049	Kestrel Ave	3.40	33	HOLD: not confirmed	JM	held
1045	Tin Harbour	3.20	215	HOLD: not confirmed	AU	held
1048	Saltmarsh Tapes	3.00	134	HOLD: not confirmed	HOLD: no country	held

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A verified artist with no payment country is held, and the country is the only reason shown.

Why: the sample's one artist with no country (Saltmarsh Tapes) is also unverified, so the data never shows a licence that passes while the country fails. A run_status that ignored country would pass the sample data.

Input:

```
INSERT into artist: artist_id=9001, stage_name=Zero Country, country=NULL, joined_date=2026-09-01, payout_rate=0.0020, verified=1
```

Expected: artist 9001 reads licence 'confirmed', payment_country 'HOLD: no country', run_status 'held'.

artist_id	stage_name	usd_per_1000_streams	days_under_licence	licence_check	payment_country	run_status
1042	Velvet Static	4.00	200	confirmed	US	ready
1044	Low Orbit Choir	3.80	239	confirmed	CA	ready
1046	Ochre Parade	3.60	172	confirmed	MX	ready
1043	Mara Quell	3.50	253	confirmed	GB	ready
1047	Juno Fleet	3.00	151	confirmed	DE	ready
1049	Kestrel Ave	3.40	33	HOLD: not confirmed	JM	held
1045	Tin Harbour	3.20	215	HOLD: not confirmed	AU	held
1048	Saltmarsh Tapes	3.00	134	HOLD: not confirmed	HOLD: no country	held
9001	Zero Country	2.00	29	confirmed	HOLD: no country	held

PASS A held artist with the highest rate in the catalogue is still listed after every ready artist.

Why: ready-first must decide the order before rate does; in the sample data no held artist out-earns them all.

Input:

```
INSERT into artist: artist_id=9002, stage_name=Top Rate Unverified, country=US, joined_date=2026-09-01, payout_rate=0.0090, verified=0
```

Expected: artist 9002 (unverified, 0.0090 per stream, the top rate) comes after all five ready artists, first among the held.

artist_id	stage_name	usd_per_1000_streams	days_under_licence	licence_check	payment_country	run_status
1042	Velvet Static	4.00	200	confirmed	US	ready
1044	Low Orbit Choir	3.80	239	confirmed	CA	ready
1046	Ochre Parade	3.60	172	confirmed	MX	ready
1043	Mara Quell	3.50	253	confirmed	GB	ready
1047	Juno Fleet	3.00	151	confirmed	DE	ready
9002	Top Rate Unverified	9.00	29	HOLD: not confirmed	US	held
1049	Kestrel Ave	3.40	33	HOLD: not confirmed	JM	held
1045	Tin Harbour	3.20	215	HOLD: not confirmed	AU	held
1048	Saltmarsh Tapes	3.00	134	HOLD: not confirmed	HOLD: no country	held

PASS Two payable artists on the same rate are ordered by artist_id.

Why: without a tie-break the order of equal rates is up to MySQL and could change between runs.

Input:

```
INSERT into artist: artist_id=900, stage_name=Aardvark Tie, country=US, joined_date=2026-01-01, payout_rate=0.0040, verified=1
```

Expected: artist 900 (rate 0.0040, same as 1042) is listed first, 1042 second.

artist_id	stage_name	usd_per_1000_streams	days_under_licence	licence_check	payment_country	run_status
900	Aardvark Tie	4.00	272	confirmed	US	ready
1042	Velvet Static	4.00	200	confirmed	US	ready
1044	Low Orbit Choir	3.80	239	confirmed	CA	ready
1046	Ochre Parade	3.60	172	confirmed	MX	ready
1043	Mara Quell	3.50	253	confirmed	GB	ready
1047	Juno Fleet	3.00	151	confirmed	DE	ready
1049	Kestrel Ave	3.40	33	HOLD: not confirmed	JM	held
1045	Tin Harbour	3.20	215	HOLD: not confirmed	AU	held
1048	Saltmarsh Tapes	3.00	134	HOLD: not confirmed	HOLD: no country	held

M2 | artist, track, play | multiple_table.sql

BUSINESS NEED

CROSSFADE pays every artist a per-stream royalty from an exact ledger: each play keeps the royalty it earned when it was recorded, so a later rate change never rewrites what is owed (RA: Overview; Play: Royalty Amount; rule I7).

QUESTION

In this royalty run, how much does each artist earn from the plays of their recordings, and how many of those plays qualified?

INPUT

The Project 1 sample data, read through these tables and columns: `crossfade.artist` (8 rows): `artist_id, stage_name`; `crossfade.track` (15 rows): `artist_id, track_id`; `crossfade.play` (31 rows): `royalty_amount, seconds_played, track_id`. Task 3 requirement met: 3.2 two or more tables joined with LEFT OUTER JOIN. One row per artist_id.

```
SELECT a.artist_id,
       a.stage_name AS payee,
       COUNT(p.track_id) AS plays,
       SUM(CASE WHEN p.seconds_played >= 30 THEN 1 ELSE 0 END) AS qualifying_plays,
       IFNULL(SUM(p.royalty_amount), 0) AS usd_owed
FROM crossfade.artist AS a
LEFT OUTER JOIN crossfade.track AS t ON t.artist_id = a.artist_id
LEFT OUTER JOIN crossfade.play AS p ON p.track_id = t.track_id
GROUP BY a.artist_id, a.stage_name
ORDER BY usd_owed DESC,
         a.artist_id;
```

EXPECTED OUTPUT: 8 row(s), in this order

artist_id	payee	plays	qualifying_plays	usd_owed
1046	Ochre Parade	6	6	0.02160
1042	Velvet Static	6	5	0.02000
1047	Juno Fleet	6	5	0.01500
1044	Low Orbit Choir	4	3	0.01140
1045	Tin Harbour	5	3	0.00960
1048	Saltmarsh Tapes	3	3	0.00900
1043	Mara Quell	1	1	0.00350
1049	Kestrel Ave	0	0	0.00000

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS An artist whose recordings nobody has played yet is listed, owed zero.

Why: the sample's one artist with no plays (1049) has no uploads either, so the second LEFT OUTER JOIN (track to play) never has to keep a track with no plays.

Input:

```
INSERT into artist: artist_id=9002, stage_name=Silent Upload, country=US, joined_date=2026-09-01, payout_rate=0.0030, verified=1
```

```
INSERT into track: track_id=9401, artist_id=9002, title=Case Track 9401, duration=200, release_date=2026-09-01,
audio_checksum=0x0000000000000000..., ingest_status=scored
```

Expected: artist 9002 with track 9401 and no plays: 0 plays, 0 qualifying, 0 owed, listed after every earning artist.

artist_id	payee	plays	qualifying_plays	usd_owed
1046	Ochre Parade	6	6	0.02160
1042	Velvet Static	6	5	0.02000
1047	Juno Fleet	6	5	0.01500
1044	Low Orbit Choir	4	3	0.01140
1045	Tin Harbour	5	3	0.00960
1048	Saltmarsh Tapes	3	3	0.00900
1043	Mara Quell	1	1	0.00350
1049	Kestrel Ave	0	0	0.00000
9002	Silent Upload	0	0	0.00000

PASS A play under 30 seconds counts as a play but not as a qualifying play (RA rule I7).

Why: checks the 30-second boundary: one play at 29 seconds, one at exactly 30.

Input:

```
INSERT into play: listener_id=20937, track_id=4829, played_at=2026-09-01 10:00:00, station_id=NULL, seconds_played=29, completed=0,
reaction=NULL, royalty_amount=0.00000
```

```
INSERT into play: listener_id=20937, track_id=4829, played_at=2026-09-02 10:00:00, station_id=NULL, seconds_played=30, completed=0,
reaction=NULL, royalty_amount=0.00350
```

Expected: artist 1048 (track 4829) gains 2 plays and 1 qualifying play; what it is owed grows by the two recorded royalties (0.00000 and 0.00350).

artist_id	payee	plays	qualifying_plays	usd_owed
1046	Ochre Parade	6	6	0.02160
1042	Velvet Static	6	5	0.02000
1047	Juno Fleet	6	5	0.01500
1048	Saltmarsh Tapes	5	4	0.01250
1044	Low Orbit Choir	4	3	0.01140
1045	Tin Harbour	5	3	0.00960
1043	Mara Quell	1	1	0.00350
1049	Kestrel Ave	0	0	0.00000

UC-02 Route payments by country; pay only verified artists

Question: Is each artist ready for this royalty run, and if not, which check is outstanding?

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`artist`	`artist_id, stage_name, country, joined_date, verified`	8 rows	alone: full

S1 | crossfade.artist | single_table.sql

BUSINESS NEED

CROSSFADE pays every artist a per-stream royalty (RA: Overview; Artist). Two artist fields bear on whether a payment can go out: Verified, "whether the service has confirmed the artist's identity and their right to licence the material they upload", and Country, used "to determine which royalty payment channels and tax treatments apply" (RA: Artist).

QUESTION

Is each artist ready for this royalty run, and if not, which check is outstanding?

INPUT

The Project 1 sample data in `crossfade.artist` (8 rows), read through these columns: `artist_id, country, joined_date, payout_rate, stage_name, verified`. Brief requirements met: (a) (c) (e).

```
SELECT artist_id,
       stage_name,
       ROUND(payout_rate * 1000, 2) AS usd_per_1000_streams,
       DATEDIFF('2026-09-30', joined_date) AS days_under_licence,
       IF(verified = 1, 'confirmed', 'HOLD: not confirmed') AS licence_check,
       IF(country IS NULL, 'HOLD: no country', country) AS payment_country,
       CASE
           WHEN verified = 1 AND country IS NOT NULL THEN 'ready'
           ELSE 'held'
       END AS run_status
FROM crossfade.artist
ORDER BY run_status DESC,
         payout_rate DESC,
         artist_id;
```

EXPECTED OUTPUT: 8 row(s), in this order

artist_id	stage_name	usd_per_1000_streams	days_under_licence	licence_check	payment_country	run_status
1042	Velvet Static	4.00	200	confirmed	US	ready
1044	Low Orbit Choir	3.80	239	confirmed	CA	ready
1046	Ochre Parade	3.60	172	confirmed	MX	ready
1043	Mara Quell	3.50	253	confirmed	GB	ready
1047	Juno Fleet	3.00	151	confirmed	DE	ready
1049	Kestrel Ave	3.40	33	HOLD: not confirmed	JM	held

artist_id	stage_name	usd_per_1000_streams	days_under_licence	licence_check	payment_country	run_status
1045	Tin Harbour	3.20	215	HOLD: not confirmed	AU	held
1048	Saltmarsh Tapes	3.00	134	HOLD: not confirmed	HOLD: no country	held

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A verified artist with no payment country is held, and the country is the only reason shown.

Why: the sample's one artist with no country (Saltmarsh Tapes) is also unverified, so the data never shows a licence that passes while the country fails. A run_status that ignored country would pass the sample data.

Input:

```
INSERT into artist: artist_id=9001, stage_name=Zero Country, country=NULL, joined_date=2026-09-01, payout_rate=0.0020, verified=1
```

Expected: artist 9001 reads licence 'confirmed', payment_country 'HOLD: no country', run_status 'held'.

artist_id	stage_name	usd_per_1000_streams	days_under_licence	licence_check	payment_country	run_status
1042	Velvet Static	4.00	200	confirmed	US	ready
1044	Low Orbit Choir	3.80	239	confirmed	CA	ready
1046	Ochre Parade	3.60	172	confirmed	MX	ready
1043	Mara Quell	3.50	253	confirmed	GB	ready
1047	Juno Fleet	3.00	151	confirmed	DE	ready
1049	Kestrel Ave	3.40	33	HOLD: not confirmed	JM	held
1045	Tin Harbour	3.20	215	HOLD: not confirmed	AU	held
1048	Saltmarsh Tapes	3.00	134	HOLD: not confirmed	HOLD: no country	held
9001	Zero Country	2.00	29	confirmed	HOLD: no country	held

PASS A held artist with the highest rate in the catalogue is still listed after every ready artist.

Why: ready-first must decide the order before rate does; in the sample data no held artist out-earns them all.

Input:

```
INSERT into artist: artist_id=9002, stage_name=Top Rate Unverified, country=US, joined_date=2026-09-01, payout_rate=0.0090, verified=0
```

Expected: artist 9002 (unverified, 0.0090 per stream, the top rate) comes after all five ready artists, first among the held.

artist_id	stage_name	usd_per_1000_streams	days_under_licence	licence_check	payment_country	run_status
1042	Velvet Static	4.00	200	confirmed	US	ready
1044	Low Orbit Choir	3.80	239	confirmed	CA	ready
1046	Ochre Parade	3.60	172	confirmed	MX	ready
1043	Mara Quell	3.50	253	confirmed	GB	ready
1047	Juno Fleet	3.00	151	confirmed	DE	ready
9002	Top Rate Unverified	9.00	29	HOLD: not confirmed	US	held
1049	Kestrel Ave	3.40	33	HOLD: not confirmed	JM	held
1045	Tin Harbour	3.20	215	HOLD: not confirmed	AU	held
1048	Saltmarsh Tapes	3.00	134	HOLD: not confirmed	HOLD: no country	held

PASS Two payable artists on the same rate are ordered by artist_id.

Why: without a tie-break the order of equal rates is up to MySQL and could change between runs.

Input:

```
INSERT into artist: artist_id=900, stage_name=Aardvark Tie, country=US, joined_date=2026-01-01, payout_rate=0.0040, verified=1
```

Expected: artist 900 (rate 0.0040, same as 1042) is listed first, 1042 second.

artist_id	stage_name	usd_per_1000_streams	days_under_licence	licence_check	payment_country	run_status
900	Aardvark Tie	4.00	272	confirmed	US	ready
1042	Velvet Static	4.00	200	confirmed	US	ready
1044	Low Orbit Choir	3.80	239	confirmed	CA	ready
1046	Ochre Parade	3.60	172	confirmed	MX	ready
1043	Mara Quell	3.50	253	confirmed	GB	ready
1047	Juno Fleet	3.00	151	confirmed	DE	ready
1049	Kestrel Ave	3.40	33	HOLD: not confirmed	JM	held
1045	Tin Harbour	3.20	215	HOLD: not confirmed	AU	held
1048	Saltmarsh Tapes	3.00	134	HOLD: not confirmed	HOLD: no country	held

UC-03 Surface new uploads as readily as hits

Question: Which are the newest recordings listeners can stream?

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`track`	`track_id, title, duration, release_date, ingest_status`	15 rows	alone: full

S2 | crossfade.track | single_table.sql

BUSINESS NEED

a recording uploaded an hour ago must be as recommendable as one with a million plays (RA: Overview), and only scored recordings may be streamed (RA: Track).

QUESTION

What are the five newest recordings listeners can stream?

INPUT

The Project 1 sample data in `crossfade.track` (15 rows), read through these columns: `duration, ingest_status, release_date, title, track_id`. Brief requirements met: (a) (c) (d).

```
SELECT track_id,
       title,
       CONCAT(duration DIV 60, ':', LPAD(MOD(duration, 60), 2, '0')) AS run_time,
       release_date,
       DATEDIFF('2026-09-30', release_date) AS days_since_release
FROM crossfade.track
WHERE ingest_status = 'scored'
ORDER BY release_date DESC,
         track_id
LIMIT 5;
```

EXPECTED OUTPUT: 5 row(s), in this order

track_id	title	run_time	release_date	days_since_release
4828	Afterimage Club	3:41	2026-07-10	82
4826	Plaza at Noon	3:18	2026-06-30	92
4829	Tapeworm Blues	3:09	2026-06-05	117
4827	Stahlwerk	4:14	2026-05-25	128
4822	Hymn for a Server Room	4:28	2026-05-15	138

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS The newest upload stays off the shelf until it is scored.

Why: only scored recordings can be streamed, so date alone must not put a track on the shelf.

Input:

```
INSERT into track: track_id=9100, artist_id=1042, title=Fresh But Unscored, duration=200, release_date=2026-09-29,
audio_checksum=0x0101010101010101..., ingest_status=awaiting scoring
```

Expected: track 9100 (released 2026-09-29, awaiting scoring) is not in the result.

track_id	title	run_time	release_date	days_since_release
4828	Afterimage Club	3:41	2026-07-10	82
4826	Plaza at Noon	3:18	2026-06-30	92
4829	Tapeworm Blues	3:09	2026-06-05	117
4827	Stahlwerk	4:14	2026-05-25	128
4822	Hymn for a Server Room	4:28	2026-05-15	138

PASS Two releases on the same day are ordered by track_id, and run time is shown as m:ss.

Why: same-day releases tie on the date; 61 seconds checks the seconds are zero-padded.

Input:

```
INSERT into track: track_id=4800, artist_id=1043, title=Same Day, duration=61, release_date=2026-07-10, audio_checksum=0x0202020202020202...,
ingest_status=scored
```

Expected: 4800 then 4828 at the top; 4800's run_time is '1:01'.

track_id	title	run_time	release_date	days_since_release
4800	Same Day	1:01	2026-07-10	82
4828	Afterimage Club	3:41	2026-07-10	82
4826	Plaza at Noon	3:18	2026-06-30	92
4829	Tapeworm Blues	3:09	2026-06-05	117
4827	Stahlwerk	4:14	2026-05-25	128

PASS The shelf never shows more than five recordings.

Why: the sample data has exactly enough scored tracks that LIMIT 5 is never pushed.

Input:

```
INSERT into track: track_id=9200, artist_id=1044, title=New 0, duration=120, release_date=2026-09-20, audio_checksum=0x1010101010101010..., ingest_status=scored
```

```
INSERT into track: track_id=9201, artist_id=1044, title=New 1, duration=120, release_date=2026-09-21, audio_checksum=0x1111111111111111..., ingest_status=scored
```

```
INSERT into track: track_id=9202, artist_id=1044, title=New 2, duration=120, release_date=2026-09-22, audio_checksum=0x1212121212121212..., ingest_status=scored
```

Expected: 5 rows, newest first: track 9202 (released 2026-09-22).

track_id	title	run_time	release_date	days_since_release
9202	New 2	2:00	2026-09-22	8
9201	New 1	2:00	2026-09-21	9
9200	New 0	2:00	2026-09-20	10
4828	Afterimage Club	3:41	2026-07-10	82
4826	Plaza at Noon	3:18	2026-06-30	92

UC-04 Track the upload pipeline; reject duplicate uploads

Question: How many recordings sit at each stage of the upload pipeline?

Checking rule I5 needs track_score. routines.sql: upload_track [P2] enters each upload as 'awaiting scoring' and refuses a duplicate checksum; record_play [P1] refuses plays of a recording that is not yet released.

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`track`	`track_id, release_date, audio_checksum, ingest_status`	15 rows	alone: partial
`track_score`	`track_id`	27 rows	with a join

S7 | crossfade.track | single_table.sql

BUSINESS NEED

every upload is scored before it can be streamed (RA: Track; its stages: Ingest Status), so a backlog in the upload pipeline must be visible.

QUESTION

How many recordings sit at each stage of the upload pipeline, and is scoring keeping up with their release dates?

INPUT

The Project 1 sample data in `crossfade.track` (15 rows), read through these columns: `ingest_status, release_date`. Brief requirements met: (a) (b) (c) (e).

```
SELECT ingest_status,
       COUNT(*) AS recordings,
       MIN(release_date) AS earliest_release,
       DATEDIFF(MIN(release_date), '2026-09-30') AS days_to_earliest_release,
       CASE
           WHEN ingest_status = 'scored' THEN 'streamable'
           WHEN ingest_status = 'rejected' THEN 'never streamable: rejected'
           WHEN MIN(release_date) <= '2026-09-30' THEN 'LATE: release date reached, still unscored'
           ELSE 'on time: release date still ahead'
       END AS release_status
FROM crossfade.track
GROUP BY ingest_status
ORDER BY ingest_status;
```

EXPECTED OUTPUT: 3 row(s), in this order

ingest_status	recordings	earliest_release	days_to_earliest_release	release_status
awaiting scoring	1	2026-09-21	-9	LATE: release date reached, still unscored
scored	13	2026-02-25	-217	streamable
rejected	1	2026-09-22	-8	never streamable: rejected

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A new rejection is counted, and statuses stay in pipeline order, not alphabetical.

Why: ingest_status is an ENUM; ORDER BY must follow awaiting scoring, scored, rejected.

Input:

```
INSERT into track: track_id=9300, artist_id=1049, title=Clipped Master, duration=30, release_date=2026-09-28,
audio_checksum=0x0303030303030303..., ingest_status=rejected
```

Expected: rejected shows 2 recordings, earliest release 2026-09-22 (8 days ago), 'never streamable: rejected'.

ingest_status	recordings	earliest_release	days_to_earliest_release	release_status
awaiting scoring	1	2026-09-21	-9	LATE: release date reached, still unscored
scored	13	2026-02-25	-217	streamable
rejected	2	2026-09-22	-8	never streamable: rejected

PASS An unscored recording whose release date is still ahead is on time, not late.

Why: the sample's one unscored recording is already past its release date (LATE), so the on-time branch is never reached.

Input:

```
UPDATE track SET release_date = '2026-10-15' WHERE ingest_status = 'awaiting scoring' (1 row(s) changed)
```

Expected: with its release moved to 2026-10-15, awaiting scoring reads 15 days ahead, 'on time: release date still ahead'.

ingest_status	recordings	earliest_release	days_to_earliest_release	release_status
awaiting scoring	1	2026-10-15	15	on time: release date still ahead
scored	13	2026-02-25	-217	streamable
rejected	1	2026-09-22	-8	never streamable: rejected

PASS A release due on the report date itself, still unscored, is already late.

Why: the boundary: on its release day the recording must already be streamable, so 'reached' includes the day.

Input:

```
UPDATE track SET release_date = '2026-09-30' WHERE ingest_status = 'awaiting scoring' (1 row(s) changed)
```

Expected: with its release moved to 2026-09-30, awaiting scoring reads 0 days, 'LATE: release date reached, still unscored'.

ingest_status	recordings	earliest_release	days_to_earliest_release	release_status
awaiting scoring	1	2026-09-30	0	LATE: release date reached, still unscored
scored	13	2026-02-25	-217	streamable
rejected	1	2026-09-22	-8	never streamable: rejected

UC-05 Show the genre legend while listeners compose a blend

Question: How wide must the composer's label column be, which genre sets that width, and how much of its one 80-character line does each description use?

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
genre	genre_id, name, description	10 rows	alone: full

S3 | crossfade.genre | single_table.sql

BUSINESS NEED

listeners compose station blends from the genre legend, which shows each genre's label and its description (RA: Genre). The RA sizes both for that display: the ten labels are fixed and "the longest, classical, has nine characters" (RA: Name), and a description is "kept to one line of 80 characters" (RA: Description).

QUESTION

How wide must the composer's label column be, which genre sets that width, and how much of its one 80-character line does each description use?

INPUT

The Project 1 sample data in `crossfade.genre` (10 rows), read through these columns: `description, genre_id, name`. Brief requirements met: (a) (b) (c) (e).

```
SELECT genre_id AS model_class_index,
       name AS label,
       CHAR_LENGTH(name) AS label_chars,
       IF(CHAR_LENGTH(name) = (SELECT MAX(CHAR_LENGTH(name)) FROM crossfade.genre),
          'sets the column width', 'fits') AS label_column,
       CHAR_LENGTH(description) AS description_chars,
       80 - CHAR_LENGTH(description) AS chars_left_on_line
FROM crossfade.genre
ORDER BY genre_id;
```

EXPECTED OUTPUT: 10 row(s), in this order

model_class_index	label	label_chars	label_column	description_chars	chars_left_on_line
0	blues	5	fits	59	21
1	classical	9	sets the column width	65	15
2	country	7	fits	57	23
3	disco	5	fits	58	22
4	hiphop	6	fits	56	24
5	jazz	4	fits	41	39
6	metal	5	fits	54	26
7	pop	3	fits	40	40
8	reggae	6	fits	45	35
9	rock	4	fits	40	40

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A genre with no description shows NULL for its description length and room left, and keeps its label row.

Why: description is the one optional genre column, and every genre in the sample data has one.

Input:

```
UPDATE genre SET description = NULL WHERE genre_id = 9 (1 row(s) changed)
```

Expected: genre 9 (rock) reads label_chars 4, 'fits', then NULL, NULL.

model_class_index	label	label_chars	label_column	description_chars	chars_left_on_line
0	blues	5	fits	59	21
1	classical	9	sets the column width	65	15
2	country	7	fits	57	23
3	disco	5	fits	58	22
4	hiphop	6	fits	56	24
5	jazz	4	fits	41	39
6	metal	5	fits	54	26
7	pop	3	fits	40	40
8	reggae	6	fits	45	35
9	rock	4	fits	NULL	NULL

PASS The genre that sets the label column width is found from the data, and genres tied on length all set it.

Why: in the sample data classical (9) is the one longest label, so a width hard-coded to 9, or a query that marks only one row, would pass.

Input:

```
UPDATE genre SET name = 'classic' WHERE genre_id = 1 (1 row(s) changed)
```

Expected: with classical renamed 'classic' (7), the widest label is 7 and both classic and country set the width.

model_class_index	label	label_chars	label_column	description_chars	chars_left_on_line
0	blues	5	fits	59	21
1	classic	7	sets the column width	65	15
2	country	7	sets the column width	57	23
3	disco	5	fits	58	22
4	hiphop	6	fits	56	24
5	jazz	4	fits	41	39
6	metal	5	fits	54	26
7	pop	3	fits	40	40
8	reggae	6	fits	45	35
9	rock	4	fits	40	40

PASS Description length is counted in characters, the unit of the RA's 80-character line, not in bytes.

Why: every sample description is plain ASCII, where characters and bytes agree; utf8mb4 stores an accented letter in two bytes, so LENGTH() would over-count.

Input:

```
UPDATE genre SET description = 'Bossa nova, samba-canção and música popular brasileira' WHERE genre_id = 5 (1 row(s) changed)
```

Expected: a jazz description with three accented letters reads 54 characters (57 bytes), 26 left on the line.

model_class_index	label	label_chars	label_column	description_chars	chars_left_on_line
0	blues	5	fits	59	21
1	classical	9	sets the column width	65	15
2	country	7	fits	57	23
3	disco	5	fits	58	22
4	hiphop	6	fits	56	24
5	jazz	4	fits	54	26
6	metal	5	fits	54	26
7	pop	3	fits	40	40
8	reggae	6	fits	45	35
9	rock	4	fits	40	40

UC-06 Bill subscribers by plan tier from their signup date

Question: What is the subscriber mix by plan tier, and how long has each tier been billing?

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`listener`	`listener_id, email, plan_tier, signup_date`	8 rows	alone: full

S4 | crossfade.listener | single_table.sql

BUSINESS NEED

subscribers are billed by plan tier, each from their own signup date (RA: Listener: Plan Tier, Signup Date).

QUESTION

How many subscribers does each plan tier have, when did the first and latest join, and how long has the average subscriber been billing?

INPUT

The Project 1 sample data in `crossfade.listener` (8 rows), read through these columns: `plan_tier, signup_date`. Brief requirements met: (a) (b) (c).

```
SELECT plan_tier,
       COUNT(*) AS listeners,
       MIN(signup_date) AS first_signup,
       MAX(signup_date) AS latest_signup,
       ROUND(AVG(DATEDIFF('2026-09-30', signup_date))) AS avg_days_subscribed
FROM crossfade.listener
GROUP BY plan_tier
ORDER BY plan_tier DESC;
```

EXPECTED OUTPUT: 3 row(s), in this order

plan_tier	listeners	first_signup	latest_signup	avg_days_subscribed
premium	2	2026-04-02	2026-05-27	154
standard	3	2026-05-09	2026-07-08	116
free	3	2026-04-18	2026-08-15	104

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A plan tier nobody is on does not appear in the subscriber mix.

Why: every tier has listeners in the sample data; GROUP BY must not invent an empty tier.

Input:

```
UPDATE listener SET plan_tier = 'standard' WHERE plan_tier = 'premium' (2 row(s) changed)
```

Expected: only standard and free, in tier order, standard with 5 listeners.

plan_tier	listeners	first_signup	latest_signup	avg_days_subscribed
standard	5	2026-04-02	2026-07-08	131
free	3	2026-04-18	2026-08-15	104

UC-07 Share stations under the creator's display name

Alone, listener only yields a list of names, which is not a real question. Station reach (creators UNION players): [M3] in multiple_table.sql. record_play [P1] in routines.sql accepts a play through another listener's station (rule I6).

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`listener`	`listener_id, display_name`	8 rows	with a join
`station`	`station_id, listener_id, name`	8 rows	with a join
`play`	`listener_id, station_id`	31 rows	with a join

M3 | station, play, listener | multiple_table.sql

BUSINESS NEED

stations can be shared, and a shared station appears under its creator's display name (RA: Listener: Display Name; Station: Name; rule I6, "A play's Station ID may refer to a station created by another listener").

QUESTION

How many listeners does each station reach, its creator included, and which stations have been shared beyond their creator?

INPUT

The Project 1 sample data, read through these tables and columns: `crossfade.station` (8 rows): `listener_id, name, station_id`; `crossfade.listener` (8 rows): `display_name, listener_id`; `crossfade.play` (31 rows): `listener_id, station_id`. Task 3 requirement met: 3.3 a set operation (UNION, INTERSECT or EXCEPT). One row per station_id.

```
SELECT s.station_id,
       s.name AS station,
       l.display_name AS created_by,
       COUNT(*) AS listeners_reached,
       CASE WHEN COUNT(*) > 1 THEN 'shared' ELSE 'creator only' END AS reach_status
FROM (SELECT station_id, listener_id
      FROM crossfade.station
      UNION
      SELECT station_id, listener_id
      FROM crossfade.play
      WHERE station_id IS NOT NULL) AS r
INNER JOIN crossfade.station AS s ON s.station_id = r.station_id
INNER JOIN crossfade.listener AS l ON l.listener_id = s.listener_id
GROUP BY s.station_id, s.name, l.display_name
ORDER BY listeners_reached DESC,
         s.station_id;
```

EXPECTED OUTPUT: 8 row(s), in this order

station_id	station	created_by	listeners_reached	reach_status
7710	Late Jazz Hop	jrivera	2	shared
7714	Harbour Dub	priyam	2	shared

station_id	station	created_by	listeners_reached	reach_status
7711	Porch and Pedal	amara_o	1	creator only
7712	Neon Waltz	amara_o	1	creator only
7713	Iron Choir	klind	1	creator only
7715	Glitterball Drift	tbarros	1	creator only
7716	Rust Belt Radio	wenz	1	creator only
7717	Blue Hour	dokafor	1	creator only

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A listener playing a station someone else created makes it shared, and equal reach orders by station_id.

Why: in the sample only two stations are shared; a third at the same reach checks the tie-break.

Input:

```
INSERT into play: listener_id=20937, track_id=4827, played_at=2026-09-01 10:00:00, station_id=7713, seconds_played=254, completed=0, reaction=NULL, royalty_amount=0.00350
```

Expected: 20937 plays station 7713 (created by 20933): 7713 reaches 2, 'shared', listed between 7710 and 7714.

station_id	station	created_by	listeners_reached	reach_status
7710	Late Jazz Hop	jrivera	2	shared
7713	Iron Choir	klind	2	shared
7714	Harbour Dub	priyam	2	shared
7711	Porch and Pedal	amara_o	1	creator only
7712	Neon Waltz	amara_o	1	creator only
7715	Glitterball Drift	tbarros	1	creator only
7716	Rust Belt Radio	wenz	1	creator only
7717	Blue Hour	dokafor	1	creator only

PASS A station nobody has played yet still reaches its creator.

Why: every sample station has been played through, so the creator half of the UNION is never the only source.

Input:

```
INSERT into station: station_id=7799, listener_id=20938, model_id=3, name=Unplayed, minimum_ambiguity=0.20, maximum_ambiguity=0.80, created_on=2026-09-01
```

Expected: station 7799 (created by 20938, no plays) is listed with 1 listener, 'creator only', created_by 'hito'.

station_id	station	created_by	listeners_reached	reach_status
7710	Late Jazz Hop	jrivera	2	shared
7714	Harbour Dub	priyam	2	shared
7711	Porch and Pedal	amara_o	1	creator only
7712	Neon Waltz	amara_o	1	creator only
7713	Iron Choir	klind	1	creator only
7715	Glitterball Drift	tbarros	1	creator only
7716	Rust Belt Radio	wenz	1	creator only
7717	Blue Hour	dokafor	1	creator only
7799	Unplayed	hito	1	creator only

UC-08 Weight each model version by its measured accuracy

Question: Which model versions should carry the most weight, and which should a new station default to?

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`scoring_model`	`model_id, version_label, fusion_type, embedding_dimension, segment_length, training_corpus, holdout_accuracy, deployed_on`	5 rows	alone: full
`station`	`station_id, model_id`	8 rows	alone: partial

S5 | crossfade.scoring_model | single_table.sql

BUSINESS NEED

each scoring model version is weighted by its measured holdout accuracy, and versions trained on different corpora are never silently compared (RA: Scoring Model).

QUESTION

Which model version should a new station default to?

INPUT

The Project 1 sample data in `crossfade.scoring_model` (5 rows), read through these columns: `deployed_on, embedding_dimension, fusion_type, holdout_accuracy, model_id, segment_length, training_corpus, version_label`. Brief requirements met: (a) (c) (d).

```
SELECT version_label,
       CONCAT(fusion_type, ' / ', embedding_dimension, 'd / ', segment_length, 's / ', training_corpus) AS architecture,
       ROUND(holdout_accuracy * 100, 2) AS accuracy_pct,
       deployed_on
FROM crossfade.scoring_model
ORDER BY holdout_accuracy DESC,
         deployed_on DESC,
         model_id
LIMIT 1;
```

EXPECTED OUTPUT: 1 row(s), in this order

version_label	architecture	accuracy_pct	deployed_on
beardown_3sec	concat / 384d / 3s / GTZAN	78.87	2026-06-23

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS On equal holdout accuracy the more recently deployed version becomes the default.

Why: the sample's top version has no tie, so the deployed_on tie-break is never used for the default.

Input:

```
INSERT into scoring_model: model_id=6, version_label=tie_newer, fusion_type=gated, embedding_dimension=256, segment_length=10,
training_corpus=GTZAN, holdout_accuracy=0.7887, deployed_on=2026-09-01
```

Expected: tie_newer (accuracy 0.7887 like beardown_3sec, but deployed later, 2026-09-01) is the one default.

version_label	architecture	accuracy_pct	deployed_on
tie_newer	gated / 256d / 10s / GTZAN	78.87	2026-09-01

PASS Two versions tied on both accuracy and deployment date: the lower model_id is the default, every time.

Why: without a last tie-break, LIMIT 1 would return whichever tied row MySQL found first.

Input:

```
INSERT into scoring_model: model_id=7, version_label=tie_high, fusion_type=concat, embedding_dimension=384, segment_length=3,
training_corpus=GTZAN, holdout_accuracy=0.9000, deployed_on=2026-09-01
```

```
INSERT into scoring_model: model_id=6, version_label=tie_low, fusion_type=concat, embedding_dimension=384, segment_length=3,
training_corpus=GTZAN, holdout_accuracy=0.9000, deployed_on=2026-09-01
```

Expected: tie_low (model_id 6) wins over tie_high (model_id 7); both at 0.9000, both deployed 2026-09-01, above every sample version. tie_high is inserted first, so insertion order cannot be what decides.

version_label	architecture	accuracy_pct	deployed_on
tie_low	concat / 384d / 3s / GTZAN	90.00	2026-09-01

S6 | crossfade.station | single_table.sql

BUSINESS NEED

every station is evaluated against one nominated model version inside an ambiguity window (RA: Station; Scoring Model 1:N Station), so retiring or redeploying a version changes the stations that rely on it.

QUESTION

How many stations does each model version serve, how wide are their ambiguity windows, and since when has each version been relied on?

INPUT

The Project 1 sample data in `crossfade.station` (8 rows), read through these columns: `created_on, maximum_ambiguity, minimum_ambiguity, model_id`. Brief requirements met: (a) (b) (c) (e).

```
SELECT model_id,
       COUNT(*) AS stations,
       ROUND(AVG(maximum_ambiguity - minimum_ambiguity), 2) AS avg_window,
       SUM(CASE WHEN maximum_ambiguity - minimum_ambiguity > 0.5 THEN 1 ELSE 0 END) AS wide_stations,
```

```
       MIN(created_on) AS oldest_station
FROM crossfade.station
GROUP BY model_id
ORDER BY stations DESC,
         model_id;
```

EXPECTED OUTPUT: 4 row(s), in this order

model_id	stations	avg_window	wide_stations	oldest_station
4	3	0.57	2	2026-06-10
2	2	0.60	2	2026-05-02
5	2	0.50	1	2026-07-20
1	1	0.60	1	2026-05-20

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS An ambiguity window of exactly 0.50 is not counted as wide.

Why: 'wide' means more than half the range; the boundary itself is not in the sample data.

Input:

```
INSERT into station: station_id=7799, listener_id=20938, model_id=3, name=Edge Window, minimum_ambiguity=0.25, maximum_ambiguity=0.75, created_on=2026-09-01
```

Expected: model 3 (only station 7799, window 0.25 to 0.75): 1 station, avg_window 0.50, 0 wide.

model_id	stations	avg_window	wide_stations	oldest_station
4	3	0.57	2	2026-06-10
2	2	0.60	2	2026-05-02
5	2	0.50	1	2026-07-20
1	1	0.60	1	2026-05-20
3	1	0.50	0	2026-09-01

UC-09 Find scoring results that predate a model redeployment

Needs track_score and scoring_model together: [M6] in multiple_table.sql lists the released recordings whose newest result predates the deployment of the newest model version.

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`track_score`	`track_id, model_id, scored_on`	27 rows	with a join
`scoring_model`	`model_id, deployed_on, version_label`	5 rows	with a join
`track`	`track_id, title, ingest_status`	15 rows	with a join

M6 | track_score, track, scoring_model | multiple_table.sql

BUSINESS NEED

several model versions are in service at once, and newer versions are deployed over time (RA: Scoring Model: the date a version "was placed in service and began scoring incoming uploads"). The RA gives Scored On one use: "to establish which results predate a model redeployment and must be recomputed" (RA: Track Score: Scored On).

QUESTION

Which released recordings have not been scored since the newest model version was deployed, and how far out of date is their newest result?

INPUT

The Project 1 sample data, read through these tables and columns: `crossfade.scoring_model` (5 rows): `deployed_on, model_id, version_label`; `crossfade.track` (15 rows): `ingest_status, title, track_id`; `crossfade.track_score` (27 rows): `model_id, scored_on, track_id`. Task 3 requirement met: 3.1 an associative table and the two tables it relates, joined with INNER JOIN. One row per track_id.

```
SELECT t.track_id,
       t.title,
       COUNT(*) AS results,
       GROUP_CONCAT(m.version_label ORDER BY m.deployed_on, m.model_id SEPARATOR ', ') AS scored_by,
       MAX(s.scored_on) AS last_scored_on,
       DATEDIFF((SELECT MAX(n.deployed_on) FROM crossfade.scoring_model AS n), MAX(s.scored_on)) AS days_out_of_date
FROM crossfade.track_score AS s
INNER JOIN crossfade.track AS t ON t.track_id = s.track_id
INNER JOIN crossfade.scoring_model AS m ON m.model_id = s.model_id
WHERE t.ingest_status = 'scored'
GROUP BY t.track_id, t.title
HAVING MAX(s.scored_on) < (SELECT MAX(n.deployed_on) FROM crossfade.scoring_model AS n)
ORDER BY days_out_of_date DESC,
         t.track_id;
```

EXPECTED OUTPUT: 9 row(s), in this order

track_id	title	results	scored_by	last_scored_on	days_out_of_date
4819	Glass Meridian	2	beardown, beardown_rrm_cfg12	2026-03-01 10:35:49	114
4821	Cathedral Static	2	beardown, beardown_rrm_cfg12	2026-03-06 16:35:42	109

track_id	title	results	scored_by	last_scored_on	days_out_of_date
4823	Dust on the Dial	2	beardown_rrm_cfg12, beardown_rrm	2026-03-24 15:58:35	91
4824	Harbour Lights Waltz	1	beardown_rrm	2026-03-24 17:19:44	91
4820	Second Weather	2	beardown_rrm_cfg12, beardown_rrm	2026-03-31 19:48:26	84
4818	Copper Lanterns	2	beardown, beardown_rrm	2026-04-26 10:47:01	58
4825	Marigold Dub	2	beardown, beardown_rrm	2026-05-07 22:13:19	47
4827	Stahlwerk	2	beardown, beardown_rrm	2026-05-31 03:30:19	23
4829	Tapeworm Blues	2	beardown_rrm_cfg12, beardown_rrm	2026-06-09 19:02:52	14

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A recording scored since the newest version was deployed no longer needs recomputing, whatever its older results.

Why: only the newest result matters; the sample's oldest stale recording (4819) is rescored here.

Input:

```
INSERT into track_score: track_id=4819, model_id=5, genre_id=1, top_probability=0.5000, posterior_entropy=0.5000, model_divergence=0.0100, segment_variance=0.0100, scored_on=2026-07-01 09:00:00
```

Expected: beardown_3sec (model 5) scores 4819 on 2026-07-01: 4819 leaves the list, which keeps the other 8.

track_id	title	results	scored_by	last_scored_on	days_out_of_date
4821	Cathedral Static	2	beardown, beardown_rrm_cfg12	2026-03-06 16:35:42	109
4823	Dust on the Dial	2	beardown_rrm_cfg12, beardown_rrm	2026-03-24 15:58:35	91
4824	Harbour Lights Waltz	1	beardown_rrm	2026-03-24 17:19:44	91
4820	Second Weather	2	beardown_rrm_cfg12, beardown_rrm	2026-03-31 19:48:26	84
4818	Copper Lanterns	2	beardown, beardown_rrm	2026-04-26 10:47:01	58
4825	Marigold Dub	2	beardown, beardown_rrm	2026-05-07 22:13:19	47
4827	Stahlwerk	2	beardown, beardown_rrm	2026-05-31 03:30:19	23
4829	Tapeworm Blues	2	beardown_rrm_cfg12, beardown_rrm	2026-06-09 19:02:52	14

PASS A result scored at midnight on the deployment day is up to date; one second earlier is not.

Why: deployed_on is a date, scored_on a moment; the comparison puts the deployment at 00:00:00 of that day.

Input:

```
INSERT into track_score: track_id=4819, model_id=5, genre_id=1, top_probability=0.5000, posterior_entropy=0.5000, model_divergence=0.0100, segment_variance=0.0100, scored_on=2026-06-23 00:00:00
```

```
INSERT into track_score: track_id=4821, model_id=5, genre_id=6, top_probability=0.5000, posterior_entropy=0.5000, model_divergence=0.0100, segment_variance=0.0100, scored_on=2026-06-22 23:59:59
```

Expected: 4819 rescored by model 5 at 2026-06-23 00:00:00 leaves the list; 4821 rescored at 2026-06-22 23:59:59 stays, 1 day out of date, now scored by 3 versions.

track_id	title	results	scored_by	last_scored_on	days_out_of_date
4823	Dust on the Dial	2	beardown_rrm_cfg12, beardown_rrm	2026-03-24 15:58:35	91
4824	Harbour Lights Waltz	1	beardown_rrm	2026-03-24 17:19:44	91
4820	Second Weather	2	beardown_rrm_cfg12, beardown_rrm	2026-03-31 19:48:26	84
4818	Copper Lanterns	2	beardown, beardown_rrm	2026-04-26 10:47:01	58
4825	Marigold Dub	2	beardown, beardown_rrm	2026-05-07 22:13:19	47
4827	Stahlwerk	2	beardown, beardown_rrm	2026-05-31 03:30:19	23
4829	Tapeworm Blues	2	beardown_rrm_cfg12, beardown_rrm	2026-06-09 19:02:52	14
4821	Cathedral Static	3	beardown, beardown_rrm_cfg12, beardown_3sec	2026-06-22 23:59:59	1

PASS Deploying a new version makes every released, scored recording due for recomputing, and equal ages order

Why: in the sample the newest version already scored 4 recordings; a version deployed after every result leaves none up to date.

Input:

```
INSERT into scoring_model: model_id=6, version_label=beardown_next, fusion_type=concat, embedding_dimension=384, segment_length=3, training_corpus=GTZAN, holdout_accuracy=0.8000, deployed_on=2026-08-01
```

Expected: version 6 deployed 2026-08-01: all 13 scored recordings are listed, the days counted from 2026-08-01.

track_id	title	results	scored_by	last_scored_on	days_out_of_date
4819	Glass Meridian	2	beardown, beardown_rrm_cfg12	2026-03-01 10:35:49	153
4821	Cathedral Static	2	beardown, beardown_rrm_cfg12	2026-03-06 16:35:42	148
4823	Dust on the Dial	2	beardown_rrm_cfg12, beardown_rrm	2026-03-24 15:58:35	130
4824	Harbour Lights Waltz	1	beardown_rrm	2026-03-24 17:19:44	130
4820	Second Weather	2	beardown_rrm_cfg12, beardown_rrm	2026-03-31 19:48:26	123
4818	Copper Lanterns	2	beardown, beardown_rrm	2026-04-26 10:47:01	97
4825	Marigold Dub	2	beardown, beardown_rrm	2026-05-07 22:13:19	86
4827	Stahlwerk	2	beardown, beardown_rrm	2026-05-31 03:30:19	62
4829	Tapeworm Blues	2	beardown_rrm_cfg12, beardown_rrm	2026-06-09 19:02:52	53
4822	Hymn for a Server Room	2	beardown_rrm, beardown_3sec	2026-06-26 05:51:40	36
4817	Low Tide Signal	4	beardown, beardown_rrm_cfg12, beardown_rrm, beardown_3sec	2026-07-01 09:14:52	31
4826	Plaza at Noon	2	beardown_rrm, beardown_3sec	2026-07-03 17:38:51	29
4828	Afterimage Club	2	beardown_rrm_cfg12, beardown_3sec	2026-07-15 10:06:28	17

PASS A rejected recording is not queued for recomputing, however old its result.

Why: none of the sample's scored results belongs to a rejected recording.

Input:

```
INSERT into track_score: track_id=4831, model_id=1, genre_id=0, top_probability=0.5000, posterior_entropy=0.5000, model_divergence=0.0100, segment_variance=0.0100, scored_on=2026-01-30 12:00:00
```

Expected: rejected 4831 scored by model 1 in January: not listed; the 9 sample rows are unchanged.

track_id	title	results	scored_by	last_scored_on	days_out_of_date
4819	Glass Meridian	2	beardown, beardown_rrm_cfg12	2026-03-01 10:35:49	114
4821	Cathedral Static	2	beardown, beardown_rrm_cfg12	2026-03-06 16:35:42	109
4823	Dust on the Dial	2	beardown_rrm_cfg12, beardown_rrm	2026-03-24 15:58:35	91
4824	Harbour Lights Waltz	1	beardown_rrm	2026-03-24 17:19:44	91
4820	Second Weather	2	beardown_rrm_cfg12, beardown_rrm	2026-03-31 19:48:26	84
4818	Copper Lanterns	2	beardown, beardown_rrm	2026-04-26 10:47:01	58
4825	Marigold Dub	2	beardown, beardown_rrm	2026-05-07 22:13:19	47
4827	Stahlwerk	2	beardown, beardown_rrm	2026-05-31 03:30:19	23
4829	Tapeworm Blues	2	beardown_rrm_cfg12, beardown_rrm	2026-06-09 19:02:52	14

UC-10 Define stations by ambiguity window; measure how long they stay in use

Question: How adventurous is each station, how long has it been in use, and which model does it rely on?

The blend weights (rule I1) live in station_blend: [M5] in multiple_table.sql fills each station from its blend and window.

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`station`	`station_id, listener_id, model_id, name, minimum_ambiguity, maximum_ambiguity, created_on`	8 rows	alone: full
`station_blend`	`station_id, genre_id, weight`	19 rows	with a join

S6 | crossfade.station | single_table.sql

BUSINESS NEED

every station is evaluated against one nominated model version inside an ambiguity window (RA: Station; Scoring Model 1:N Station), so retiring or redeploying a version changes the stations that rely on it.

QUESTION

How many stations does each model version serve, how wide are their ambiguity windows, and since when has each version been relied on?

INPUT

The Project 1 sample data in `crossfade.station` (8 rows), read through these columns: `created_on, maximum_ambiguity, minimum_ambiguity, model_id`. Brief requirements met: (a) (b) (c) (e).

```
SELECT model_id,
       COUNT(*) AS stations,
       ROUND(AVG(maximum_ambiguity - minimum_ambiguity), 2) AS avg_window,
       SUM(CASE WHEN maximum_ambiguity - minimum_ambiguity > 0.5 THEN 1 ELSE 0 END) AS wide_stations,
       MIN(created_on) AS oldest_station
FROM crossfade.station
GROUP BY model_id
ORDER BY stations DESC,
         model_id;
```

EXPECTED OUTPUT: 4 row(s), in this order

model_id	stations	avg_window	wide_stations	oldest_station
4	3	0.57	2	2026-06-10
2	2	0.60	2	2026-05-02
5	2	0.50	1	2026-07-20
1	1	0.60	1	2026-05-20

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS An ambiguity window of exactly 0.50 is not counted as wide.

Why: 'wide' means more than half the range; the boundary itself is not in the sample data.

Input:

```
INSERT into station: station_id=7799, listener_id=20938, model_id=3, name=Edge Window, minimum_ambiguity=0.25, maximum_ambiguity=0.75, created_on=2026-09-01
```

Expected: model 3 (only station 7799, window 0.25 to 0.75): 1 station, avg_window 0.50, 0 wide.

model_id	stations	avg_window	wide_stations	oldest_station
4	3	0.57	2	2026-06-10
2	2	0.60	2	2026-05-02
5	2	0.50	1	2026-07-20
1	1	0.60	1	2026-05-20
3	1	0.50	0	2026-09-01

M5 | station, track_score, track, station_blend, genre | multiple_table.sql

BUSINESS NEED

a station stores a genre blend and an ambiguity window, not a fixed list, and the service "returns recordings that genuinely occupy that region" (RA: Overview; Station). Which recordings belong on a station has to be worked out from the business plan's own rules, so the query applies each of them in turn: 1. only scored recordings can be streamed (RA: Track); 2. a station is evaluated against its one nominated model version, "so that a station's contents do not change meaning when a different version scores the catalogue" (RA: Scoring Model 1:N Station); 3. the recording's posterior entropy must lie inside the station's window: the minimum excludes recordings "that sit squarely inside a single genre", the maximum those "the models cannot place with any confidence" (RA: Minimum and Maximum Ambiguity); 4. its top genre must be one of the station's blend genres (RA: Station Blend). The RA stores the top genre and the entropy on the scoring result "so that stations can filter without reading ten rows per score" (RA: Track Score), so the filter reads track_score, not the ten genre_probability rows. The RA does not rank qualifying recordings, so the query orders them by the station's own preference: the blend weight of their top genre, highest first (RA: Weight, "the share of the station's blend given to this genre"), then the higher posterior entropy, the recording further between genres, which is the service's selling point; track_id breaks ties.

QUESTION

For each station, which recordings qualify under its model, window and blend, in what order, and how long would the station run?

INPUT

The Project 1 sample data, read through these tables and columns: `crossfade.track` (15 rows): `duration, ingest_status, title, track_id`; `crossfade.station` (8 rows): `maximum_ambiguity, minimum_ambiguity, model_id, name, station_id`; `crossfade.track_score` (27 rows): `genre_id, model_id, posterior_entropy, track_id`; `crossfade.genre` (10 rows): `genre_id, name`; `crossfade.station_blend` (19 rows): `genre_id, station_id, weight`. Task 3 requirement met: 3.5 a derived table defined with WITH. One row per station_id, track_id.

```
WITH eligible AS (
    SELECT st.station_id,
           st.name AS station,
```

```
           t.track_id,
           t.title,
           t.duration,
           g.name AS top_genre,
           b.weight AS genre_weight,
           s.posterior_entropy
    FROM crossfade.station AS st
    INNER JOIN crossfade.track_score AS s ON s.model_id = st.model_id
    INNER JOIN crossfade.track AS t ON t.track_id = s.track_id
    INNER JOIN crossfade.station_blend AS b ON b.station_id = st.station_id
                                           AND b.genre_id = s.genre_id
    INNER JOIN crossfade.genre AS g ON g.genre_id = s.genre_id
    WHERE t.ingest_status = 'scored'
      AND s.posterior_entropy BETWEEN st.minimum_ambiguity AND st.maximum_ambiguity
),
station_length AS (
    SELECT station_id,
           COUNT(*) AS recordings,
           ROUND(SUM(duration) / 60, 1) AS run_minutes
    FROM eligible
    GROUP BY station_id
)
SELECT e.station_id,
       e.station,
       e.track_id,
       e.title,
       e.top_genre,
       e.genre_weight,
       e.posterior_entropy,
       sl.recordings AS station_recordings,
       sl.run_minutes AS station_run_minutes
FROM eligible AS e
INNER JOIN station_length AS sl ON sl.station_id = e.station_id
ORDER BY e.station_id,
         e.genre_weight DESC,
         e.posterior_entropy DESC,
         e.track_id;
```

EXPECTED OUTPUT: 20 row(s), in this order

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7710	Late Jazz Hop	4817	Low Tide Signal	jazz	0.60	0.4619	1	3.6
7711	Porch and Pedal	4823	Dust on the Dial	country	0.70	0.5065	4	12.6
7711	Porch and Pedal	4824	Harbour Lights Waltz	country	0.70	0.4276	4	12.6
7711	Porch and Pedal	4829	Tapeworm Blues	blues	0.30	0.4989	4	12.6
7711	Porch and Pedal	4818	Copper Lanterns	blues	0.30	0.4628	4	12.6
7712	Neon Waltz	4828	Afterimage Club	pop	0.50	0.5261	4	16.2

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7712	Neon Waltz	4820	Second Weather	pop	0.50	0.5158	4	16.2
7712	Neon Waltz	4821	Cathedral Static	classical	0.20	0.4998	4	16.2
7712	Neon Waltz	4819	Glass Meridian	classical	0.20	0.4114	4	16.2
7713	Iron Choir	4821	Cathedral Static	metal	0.50	0.5700	3	13.5
7713	Iron Choir	4827	Stahlwerk	metal	0.50	0.4784	3	13.5
7713	Iron Choir	4819	Glass Meridian	classical	0.34	0.5310	3	13.5
7714	Harbour Dub	4825	Marigold Dub	reggae	0.60	0.6107	1	3.9
7715	Glitterball Drift	4826	Plaza at Noon	disco	0.34	0.5503	2	7.0
7715	Glitterball Drift	4828	Afterimage Club	disco	0.34	0.5417	2	7.0
7716	Rust Belt Radio	4829	Tapeworm Blues	blues	0.25	0.4989	4	12.6
7716	Rust Belt Radio	4818	Copper Lanterns	blues	0.25	0.4628	4	12.6
7716	Rust Belt Radio	4823	Dust on the Dial	country	0.20	0.5065	4	12.6
7716	Rust Belt Radio	4824	Harbour Lights Waltz	country	0.20	0.4276	4	12.6
7717	Blue Hour	4829	Tapeworm Blues	blues	1.00	0.5220	1	3.2

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A recording whose entropy equals the window's minimum or maximum is on the station; one just outside is not.

Why: no sample score falls exactly on a window edge, so BETWEEN's inclusive edges are never tested.

Input:

```
INSERT into station: station_id=7798, listener_id=20938, model_id=3, name=Edges, minimum_ambiguity=0.40, maximum_ambiguity=0.60,
created_on=2026-09-01

INSERT into station_blend: station_id=7798, genre_id=2, weight=1.00

INSERT into track_score: track_id=4817, model_id=3, genre_id=2, top_probability=0.5000, posterior_entropy=0.4000, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00

INSERT into track_score: track_id=4818, model_id=3, genre_id=2, top_probability=0.5000, posterior_entropy=0.6000, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00

INSERT into track_score: track_id=4819, model_id=3, genre_id=2, top_probability=0.5000, posterior_entropy=0.3999, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00

INSERT into track_score: track_id=4820, model_id=3, genre_id=2, top_probability=0.5000, posterior_entropy=0.6001, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00
```

Expected: station 7798 (model 3, window 0.40 to 0.60, blend country 1.00) with four model-3 country scores: 4818 (0.6000) and 4817 (0.4000) are on it, higher entropy first; 4819 (0.3999) and 4820 (0.6001) are not.

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7710	Late Jazz Hop	4817	Low Tide Signal	jazz	0.60	0.4619	1	3.6
7711	Porch and Pedal	4823	Dust on the Dial	country	0.70	0.5065	4	12.6
7711	Porch and Pedal	4824	Harbour Lights Waltz	country	0.70	0.4276	4	12.6
7711	Porch and Pedal	4829	Tapeworm Blues	blues	0.30	0.4989	4	12.6
7711	Porch and Pedal	4818	Copper Lanterns	blues	0.30	0.4628	4	12.6
7712	Neon Waltz	4828	Afterimage Club	pop	0.50	0.5261	4	16.2
7712	Neon Waltz	4820	Second Weather	pop	0.50	0.5158	4	16.2
7712	Neon Waltz	4821	Cathedral Static	classical	0.20	0.4998	4	16.2
7712	Neon Waltz	4819	Glass Meridian	classical	0.20	0.4114	4	16.2
7713	Iron Choir	4821	Cathedral Static	metal	0.50	0.5700	3	13.5
7713	Iron Choir	4827	Stahlwerk	metal	0.50	0.4784	3	13.5
7713	Iron Choir	4819	Glass Meridian	classical	0.34	0.5310	3	13.5
7714	Harbour Dub	4825	Marigold Dub	reggae	0.60	0.6107	1	3.9
7715	Glitterball Drift	4826	Plaza at Noon	disco	0.34	0.5503	2	7.0
7715	Glitterball Drift	4828	Afterimage Club	disco	0.34	0.5417	2	7.0
7716	Rust Belt Radio	4829	Tapeworm Blues	blues	0.25	0.4989	4	12.6
7716	Rust Belt Radio	4818	Copper Lanterns	blues	0.25	0.4628	4	12.6
7716	Rust Belt Radio	4823	Dust on the Dial	country	0.20	0.5065	4	12.6
7716	Rust Belt Radio	4824	Harbour Lights Waltz	country	0.20	0.4276	4	12.6
7717	Blue Hour	4829	Tapeworm Blues	blues	1.00	0.5220	1	3.2

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7798	Edges	4818	Copper Lanterns	country	1.00	0.6000	2	6.7
7798	Edges	4817	Low Tide Signal	country	1.00	0.4000	2	6.7

PASS A rejected recording stays off every station even if a model scored it.

Why: every scored row in the sample belongs to a scored recording, so the ingest-status rule is never needed.

Input:

```
INSERT into track_score: track_id=4831, model_id=2, genre_id=0, top_probability=0.5000, posterior_entropy=0.5000, model_divergence=0.0100, segment_variance=0.0100, scored_on=2026-09-01 12:00:00
```

Expected: rejected track 4831, scored by model 2 as genre 0 at entropy 0.5000 (inside station 7717's window and blend), is not on 7717, which still holds 1 recording.

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7710	Late Jazz Hop	4817	Low Tide Signal	jazz	0.60	0.4619	1	3.6
7711	Porch and Pedal	4823	Dust on the Dial	country	0.70	0.5065	4	12.6
7711	Porch and Pedal	4824	Harbour Lights Waltz	country	0.70	0.4276	4	12.6
7711	Porch and Pedal	4829	Tapeworm Blues	blues	0.30	0.4989	4	12.6
7711	Porch and Pedal	4818	Copper Lanterns	blues	0.30	0.4628	4	12.6
7712	Neon Waltz	4828	Afterimage Club	pop	0.50	0.5261	4	16.2
7712	Neon Waltz	4820	Second Weather	pop	0.50	0.5158	4	16.2
7712	Neon Waltz	4821	Cathedral Static	classical	0.20	0.4998	4	16.2
7712	Neon Waltz	4819	Glass Meridian	classical	0.20	0.4114	4	16.2
7713	Iron Choir	4821	Cathedral Static	metal	0.50	0.5700	3	13.5
7713	Iron Choir	4827	Stahlwerk	metal	0.50	0.4784	3	13.5
7713	Iron Choir	4819	Glass Meridian	classical	0.34	0.5310	3	13.5
7714	Harbour Dub	4825	Marigold Dub	reggae	0.60	0.6107	1	3.9
7715	Glitterball Drift	4826	Plaza at Noon	disco	0.34	0.5503	2	7.0
7715	Glitterball Drift	4828	Afterimage Club	disco	0.34	0.5417	2	7.0
7716	Rust Belt Radio	4829	Tapeworm Blues	blues	0.25	0.4989	4	12.6
7716	Rust Belt Radio	4818	Copper Lanterns	blues	0.25	0.4628	4	12.6
7716	Rust Belt Radio	4823	Dust on the Dial	country	0.20	0.5065	4	12.6
7716	Rust Belt Radio	4824	Harbour Lights Waltz	country	0.20	0.4276	4	12.6
7717	Blue Hour	4829	Tapeworm Blues	blues	1.00	0.5220	1	3.2

PASS A station holds only recordings whose top genre is in its blend, as placed by its own model version.

Why: a new station with a one-genre blend and the widest window isolates the blend and model rules.

Input:

```
INSERT into station: station_id=7797, listener_id=20938, model_id=4, name=One Genre, minimum_ambiguity=0.00, maximum_ambiguity=1.00, created_on=2026-09-01
```

```
INSERT into station_blend: station_id=7797, genre_id=8, weight=1.00
```

Expected: station 7797 (model 4, window 0 to 1, blend genre 8 only) holds just 4825 (model 4 places it in genre 8; model 1 also does, but model 1 is not this station's model): 1 recording, 4825's duration in minutes.

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7710	Late Jazz Hop	4817	Low Tide Signal	jazz	0.60	0.4619	1	3.6
7711	Porch and Pedal	4823	Dust on the Dial	country	0.70	0.5065	4	12.6
7711	Porch and Pedal	4824	Harbour Lights Waltz	country	0.70	0.4276	4	12.6
7711	Porch and Pedal	4829	Tapeworm Blues	blues	0.30	0.4989	4	12.6
7711	Porch and Pedal	4818	Copper Lanterns	blues	0.30	0.4628	4	12.6
7712	Neon Waltz	4828	Afterimage Club	pop	0.50	0.5261	4	16.2
7712	Neon Waltz	4820	Second Weather	pop	0.50	0.5158	4	16.2
7712	Neon Waltz	4821	Cathedral Static	classical	0.20	0.4998	4	16.2
7712	Neon Waltz	4819	Glass Meridian	classical	0.20	0.4114	4	16.2
7713	Iron Choir	4821	Cathedral Static	metal	0.50	0.5700	3	13.5
7713	Iron Choir	4827	Stahlwerk	metal	0.50	0.4784	3	13.5
7713	Iron Choir	4819	Glass Meridian	classical	0.34	0.5310	3	13.5
7714	Harbour Dub	4825	Marigold Dub	reggae	0.60	0.6107	1	3.9
7715	Glitterball Drift	4826	Plaza at Noon	disco	0.34	0.5503	2	7.0
7715	Glitterball Drift	4828	Afterimage Club	disco	0.34	0.5417	2	7.0
7716	Rust Belt Radio	4829	Tapeworm Blues	blues	0.25	0.4989	4	12.6
7716	Rust Belt Radio	4818	Copper Lanterns	blues	0.25	0.4628	4	12.6
7716	Rust Belt Radio	4823	Dust on the Dial	country	0.20	0.5065	4	12.6
7716	Rust Belt Radio	4824	Harbour Lights Waltz	country	0.20	0.4276	4	12.6
7717	Blue Hour	4829	Tapeworm Blues	blues	1.00	0.5220	1	3.2
7797	One Genre	4825	Marigold Dub	reggae	1.00	0.6107	1	3.9

UC-11 Place each recording between genres

Lives in the associative tables track_score and genre_probability: [M4] in multiple_table.sql finds recordings the model versions disagree on, and [M7] reads each result's full distribution for its runner-up genre and how close it is.

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`track_score`	`track_id, model_id, genre_id, top_probability, posterior_entropy, model_divergence, segment_variance`	27 rows	with a join
`genre_probability`	`track_id, model_id, genre_id, probability`	270 rows	with a join
`genre`	`genre_id, name`	10 rows	with a join
`track`	`track_id, title`	15 rows	with a join

M4 | track, track_score | multiple_table.sql

BUSINESS NEED

CROSSFADE places every recording between genres; each model version returns a distribution with measures of how uncertain and how inconsistent it is, and model divergence records how far one version sits from the others that scored the same recording (RA: Overview; Track Score: Model Divergence).

QUESTION

Which recordings do the model versions place in different top genres, and how far apart are their distributions?

INPUT

The Project 1 sample data, read through these tables and columns: `crossfade.track` (15 rows): `title, track_id`; `crossfade.track_score` (27 rows): `genre_id, model_divergence, track_id`. Task 3 requirement met: 3.4 a subquery with a multi-row operation (IN, NOT IN, ANY, SOME, ALL or EXISTS). One row per track_id.

```
SELECT t.track_id,
       t.title,
       (SELECT COUNT(*)
        FROM crossfade.track_score AS sc
        WHERE sc.track_id = t.track_id) AS models_scoring,
       (SELECT MAX(sd.model_divergence)
        FROM crossfade.track_score AS sd
        WHERE sd.track_id = t.track_id) AS highest_divergence
FROM crossfade.track AS t
WHERE t.track_id IN (SELECT s.track_id
                     FROM crossfade.track_score AS s
                     GROUP BY s.track_id
                     HAVING COUNT(DISTINCT s.genre_id) > 1)
ORDER BY highest_divergence DESC,
         t.track_id;
```

EXPECTED OUTPUT: 3 row(s), in this order

track_id	title	models_scoring	highest_divergence
4821	Cathedral Static	2	0.1025

track_id	title	models_scoring	highest_divergence
4828	Afterimage Club	2	0.0421
4818	Copper Lanterns	2	0.0283

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A second model version that names a different top genre puts the recording on the list; equal divergence

Why: 4824 is scored by one version only in the sample, so it can never qualify; a disagreeing score with the same divergence as 4828 (0.0421) checks the tie-break too.

Input:

```
INSERT into track_score: track_id=4824, model_id=1, genre_id=9, top_probability=0.5000, posterior_entropy=0.4500, model_divergence=0.0421, segment_variance=0.0100, scored_on=2026-09-01 12:00:00
```

Expected: 4824 joins with 2 models scoring, listed just before 4828.

track_id	title	models_scoring	highest_divergence
4821	Cathedral Static	2	0.1025
4824	Harbour Lights Waltz	2	0.0421
4828	Afterimage Club	2	0.0421
4818	Copper Lanterns	2	0.0283

PASS With three versions scoring a recording, the highest of their divergences is reported.

Why: every listed sample recording is scored by exactly two versions, whose divergences from each other are the same number, so MAX and MIN agree and a wrong aggregate would go unnoticed.

Input:

```
INSERT into track_score: track_id=4821, model_id=4, genre_id=6, top_probability=0.5000, posterior_entropy=0.5500, model_divergence=0.2000, segment_variance=0.0100, scored_on=2026-09-01 12:00:00
```

Expected: model 4 also scores 4821 (metal, divergence 0.2000): 4821 shows 3 models scoring, highest 0.2000.

track_id	title	models_scoring	highest_divergence
4821	Cathedral Static	3	0.2000
4828	Afterimage Club	2	0.0421
4818	Copper Lanterns	2	0.0283

PASS Another version that agrees on the top genre does not make a disagreement.

Why: the IN subquery must count distinct top genres, not scores.

Input:

```
INSERT into track_score: track_id=4824, model_id=1, genre_id=2, top_probability=0.5000, posterior_entropy=0.4500, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00
```

Expected: model 1 scores 4824 as genre 2, like model 4 did: the result is unchanged (3 recordings).

track_id	title	models_scoring	highest_divergence
4821	Cathedral Static	2	0.1025
4828	Afterimage Club	2	0.0421
4818	Copper Lanterns	2	0.0283

PASS A disagreement whose divergence is not recorded is still listed, after every measured one.

Why: model_divergence may be NULL; MAX of NULLs is NULL, and MySQL sorts NULL last in a DESC order.

Input:

```
INSERT into track: track_id=9500, artist_id=1042, title=Case Track 9500, duration=200, release_date=2026-09-01,
audio_checksum=0x0000000000000000..., ingest_status=scored
```

```
INSERT into track_score: track_id=9500, model_id=1, genre_id=9, top_probability=0.5000, posterior_entropy=0.6000, model_divergence=NULL,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00
```

```
INSERT into track_score: track_id=9500, model_id=2, genre_id=5, top_probability=0.5000, posterior_entropy=0.6000, model_divergence=NULL,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00
```

Expected: new recording 9500, scored rock by model 1 and jazz by model 2, both without divergence: listed last, highest_divergence NULL.

track_id	title	models_scoring	highest_divergence
4821	Cathedral Static	2	0.1025
4828	Afterimage Club	2	0.0421
4818	Copper Lanterns	2	0.0283
9500	Case Track 9500	2	NULL

M7 | genre_probability, track_score, track, genre | multiple_table.sql

BUSINESS NEED

the catalogue "is indexed by where a recording sits between genres rather than by a single genre label" (RA: Overview). Genre Probability "holds the full distribution that stations match against, which the single top genre on Track Score cannot express" (RA: Genre Probability), so placing a recording between genres means reading past its top genre to the genre it sits closest to next.

QUESTION

For each scoring result, which genre is the runner-up to the top genre, how close is it, and does the recording change character partway through?

INPUT

The Project 1 sample data, read through these tables and columns: `crossfade.track_score` (27 rows): `genre_id, model_id, segment_variance, top_probability, track_id`; `crossfade.genre` (10 rows): `genre_id, name`; `crossfade.track` (15 rows): `title, track_id`; `crossfade.genre_probability` (270 rows): `genre_id, model_id, probability, track_id`. Task 3 requirement met: 3.5 a derived table defined with WITH. One row per track_id, model_id.

```
WITH second_place AS (
    SELECT gp.track_id,
           gp.model_id,
           MAX(gp.probability) AS probability
    FROM crossfade.genre_probability AS gp
    INNER JOIN crossfade.track_score AS s ON s.track_id = gp.track_id
                                         AND s.model_id = gp.model_id
    WHERE gp.genre_id <> s.genre_id
    GROUP BY gp.track_id, gp.model_id
),
runner_up AS (
    SELECT gp.track_id,
           gp.model_id,
           MIN(gp.genre_id) AS genre_id,
           sp.probability
    FROM crossfade.genre_probability AS gp
    INNER JOIN second_place AS sp ON sp.track_id = gp.track_id
                                 AND sp.model_id = gp.model_id
                                 AND sp.probability = gp.probability
    INNER JOIN crossfade.track_score AS s ON s.track_id = gp.track_id
                                         AND s.model_id = gp.model_id
    WHERE gp.genre_id <> s.genre_id
    GROUP BY gp.track_id, gp.model_id, sp.probability
)
SELECT t.track_id,
       t.title,
       s.model_id,
       g1.name AS top_genre,
       s.top_probability,
       g2.name AS runner_up_genre,
       r.probability AS runner_up_probability,
       s.top_probability - r.probability AS margin,
       s.segment_variance
FROM crossfade.track_score AS s
INNER JOIN crossfade.track AS t ON t.track_id = s.track_id
INNER JOIN runner_up AS r ON r.track_id = s.track_id
                         AND r.model_id = s.model_id
INNER JOIN crossfade.genre AS g1 ON g1.genre_id = s.genre_id
INNER JOIN crossfade.genre AS g2 ON g2.genre_id = r.genre_id
ORDER BY margin,
         t.track_id,
         s.model_id;
```

EXPECTED OUTPUT: 27 row(s), in this order

track_id	title	model_id	top_genre	top_probability	runner_up_genre	runner_up_probability	margin	segment_variance
4820	Second Weather	2	pop	0.4357	disco	0.4304	0.0053	0.0057
4828	Afterimage Club	2	pop	0.4461	disco	0.4273	0.0188	0.0054
4827	Stahlwerk	4	metal	0.4514	rock	0.3636	0.0878	0.0072
4820	Second Weather	4	pop	0.4671	disco	0.3764	0.0907	0.0256
4818	Copper Lanterns	4	blues	0.5077	jazz	0.4133	0.0944	0.0109
4818	Copper Lanterns	1	jazz	0.4819	blues	0.3774	0.1045	0.0111
4826	Plaza at Noon	4	disco	0.4338	reggae	0.3212	0.1126	0.0180
4819	Glass Meridian	1	classical	0.4890	pop	0.3745	0.1145	0.0105
4829	Tapeworm Blues	2	blues	0.4947	rock	0.3777	0.1170	0.0265
4821	Cathedral Static	1	metal	0.4799	classical	0.3612	0.1187	0.0081
4826	Plaza at Noon	5	disco	0.4943	reggae	0.3632	0.1311	0.0193
4823	Dust on the Dial	2	country	0.5153	rock	0.3570	0.1583	0.0128
4819	Glass Meridian	2	classical	0.5562	pop	0.3888	0.1674	0.0233
4829	Tapeworm Blues	4	blues	0.5363	rock	0.3526	0.1837	0.0036
4817	Low Tide Signal	1	jazz	0.5438	hiphop	0.3426	0.2012	0.0243
4828	Afterimage Club	5	disco	0.5516	pop	0.3023	0.2493	0.0246
4825	Marigold Dub	1	reggae	0.4964	hiphop	0.2332	0.2632	0.0273
4821	Cathedral Static	2	classical	0.5556	metal	0.2865	0.2691	0.0288
4817	Low Tide Signal	5	jazz	0.5800	hiphop	0.2843	0.2957	0.0051
4817	Low Tide Signal	4	jazz	0.6002	hiphop	0.2859	0.3143	0.0289
4817	Low Tide Signal	2	jazz	0.6131	hiphop	0.2898	0.3233	0.0058
4823	Dust on the Dial	4	country	0.6131	rock	0.2525	0.3606	0.0179
4825	Marigold Dub	4	reggae	0.5743	hiphop	0.1940	0.3803	0.0246
4827	Stahlwerk	1	metal	0.6295	rock	0.2481	0.3814	0.0117
4822	Hymn for a Server Room	5	classical	0.6804	jazz	0.1499	0.5305	0.0121
4822	Hymn for a Server Room	4	classical	0.7354	jazz	0.1269	0.6085	0.0276
4824	Harbour Lights Waltz	4	country	0.7401	blues	0.1002	0.6399	0.0089

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS When two genres share the runner-up probability, the lower genre_id is the runner-up, and the result

Why: no sample distribution has a tie for second place; without the rule the result would appear twice.

Input:

```
INSERT into track_score: track_id=4824, model_id=1, genre_id=2, top_probability=0.4000, posterior_entropy=0.6000, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00
INSERT into genre_probability: track_id=4824, model_id=1, genre_id=0, probability=0.0286
INSERT into genre_probability: track_id=4824, model_id=1, genre_id=1, probability=0.0286
INSERT into genre_probability: track_id=4824, model_id=1, genre_id=2, probability=0.4000
INSERT into genre_probability: track_id=4824, model_id=1, genre_id=3, probability=0.2000
INSERT into genre_probability: track_id=4824, model_id=1, genre_id=4, probability=0.0286
INSERT into genre_probability: track_id=4824, model_id=1, genre_id=5, probability=0.2000
INSERT into genre_probability: track_id=4824, model_id=1, genre_id=6, probability=0.0286
INSERT into genre_probability: track_id=4824, model_id=1, genre_id=7, probability=0.0286
INSERT into genre_probability: track_id=4824, model_id=1, genre_id=8, probability=0.0286
INSERT into genre_probability: track_id=4824, model_id=1, genre_id=9, probability=0.0286
```

Expected: model 1 scores 4824 as country (2, 0.4000) with disco (3) and jazz (5) both at 0.2000: one row, runner-up disco, margin 0.2000.

track_id	title	model_id	top_genre	top_probability	runner_up_genre	runner_up_probability	margin	segment_variance
4820	Second Weather	2	pop	0.4357	disco	0.4304	0.0053	0.0057
4828	Afterimage Club	2	pop	0.4461	disco	0.4273	0.0188	0.0054
4827	Stahlwerk	4	metal	0.4514	rock	0.3636	0.0878	0.0072
4820	Second Weather	4	pop	0.4671	disco	0.3764	0.0907	0.0256
4818	Copper Lanterns	4	blues	0.5077	jazz	0.4133	0.0944	0.0109
4818	Copper Lanterns	1	jazz	0.4819	blues	0.3774	0.1045	0.0111
4826	Plaza at Noon	4	disco	0.4338	reggae	0.3212	0.1126	0.0180
4819	Glass Meridian	1	classical	0.4890	pop	0.3745	0.1145	0.0105
4829	Tapeworm Blues	2	blues	0.4947	rock	0.3777	0.1170	0.0265
4821	Cathedral Static	1	metal	0.4799	classical	0.3612	0.1187	0.0081
4826	Plaza at Noon	5	disco	0.4943	reggae	0.3632	0.1311	0.0193
4823	Dust on the Dial	2	country	0.5153	rock	0.3570	0.1583	0.0128
4819	Glass Meridian	2	classical	0.5562	pop	0.3888	0.1674	0.0233
4829	Tapeworm Blues	4	blues	0.5363	rock	0.3526	0.1837	0.0036
4824	Harbour Lights Waltz	1	country	0.4000	disco	0.2000	0.2000	0.0100
4817	Low Tide Signal	1	jazz	0.5438	hiphop	0.3426	0.2012	0.0243
4828	Afterimage Club	5	disco	0.5516	pop	0.3023	0.2493	0.0246

track_id	title	model_id	top_genre	top_probability	runner_up_genre	runner_up_probability	margin	segment_variance
4825	Marigold Dub	1	reggae	0.4964	hiphop	0.2332	0.2632	0.0273
4821	Cathedral Static	2	classical	0.5556	metal	0.2865	0.2691	0.0288
4817	Low Tide Signal	5	jazz	0.5800	hiphop	0.2843	0.2957	0.0051
4817	Low Tide Signal	4	jazz	0.6002	hiphop	0.2859	0.3143	0.0289
4817	Low Tide Signal	2	jazz	0.6131	hiphop	0.2898	0.3233	0.0058
4823	Dust on the Dial	4	country	0.6131	rock	0.2525	0.3606	0.0179
4825	Marigold Dub	4	reggae	0.5743	hiphop	0.1940	0.3803	0.0246
4827	Stahlwerk	1	metal	0.6295	rock	0.2481	0.3814	0.0117
4822	Hymn for a Server Room	5	classical	0.6804	jazz	0.1499	0.5305	0.0121
4822	Hymn for a Server Room	4	classical	0.7354	jazz	0.1269	0.6085	0.0276
4824	Harbour Lights Waltz	4	country	0.7401	blues	0.1002	0.6399	0.0089

PASS When two genres share the top probability, the other one is the runner-up and the margin is 0: the

Why: the sample has no tied top genre; margin 0 is the strongest "between genres" case.

Input:

```
INSERT into track_score: track_id=4824, model_id=2, genre_id=7, top_probability=0.4000, posterior_entropy=0.7000, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00

INSERT into genre_probability: track_id=4824, model_id=2, genre_id=0, probability=0.0250

INSERT into genre_probability: track_id=4824, model_id=2, genre_id=1, probability=0.0250

INSERT into genre_probability: track_id=4824, model_id=2, genre_id=2, probability=0.0250

INSERT into genre_probability: track_id=4824, model_id=2, genre_id=3, probability=0.0250

INSERT into genre_probability: track_id=4824, model_id=2, genre_id=4, probability=0.0250

INSERT into genre_probability: track_id=4824, model_id=2, genre_id=5, probability=0.0250

INSERT into genre_probability: track_id=4824, model_id=2, genre_id=6, probability=0.0250

INSERT into genre_probability: track_id=4824, model_id=2, genre_id=7, probability=0.4000

INSERT into genre_probability: track_id=4824, model_id=2, genre_id=8, probability=0.0250

INSERT into genre_probability: track_id=4824, model_id=2, genre_id=9, probability=0.4000
```

Expected: model 2 scores 4824 with pop (7) and rock (9) both at 0.4000 and pop as its top genre: runner-up rock, margin 0.0000, first row.

track_id	title	model_id	top_genre	top_probability	runner_up_genre	runner_up_probability	margin	segment_variance
4824	Harbour Lights Waltz	2	pop	0.4000	rock	0.4000	0.0000	0.0100
4820	Second Weather	2	pop	0.4357	disco	0.4304	0.0053	0.0057
4828	Afterimage Club	2	pop	0.4461	disco	0.4273	0.0188	0.0054
4827	Stahlwerk	4	metal	0.4514	rock	0.3636	0.0878	0.0072
4820	Second Weather	4	pop	0.4671	disco	0.3764	0.0907	0.0256
4818	Copper Lanterns	4	blues	0.5077	jazz	0.4133	0.0944	0.0109
4818	Copper Lanterns	1	jazz	0.4819	blues	0.3774	0.1045	0.0111
4826	Plaza at Noon	4	disco	0.4338	reggae	0.3212	0.1126	0.0180
4819	Glass Meridian	1	classical	0.4890	pop	0.3745	0.1145	0.0105
4829	Tapeworm Blues	2	blues	0.4947	rock	0.3777	0.1170	0.0265
4821	Cathedral Static	1	metal	0.4799	classical	0.3612	0.1187	0.0081
4826	Plaza at Noon	5	disco	0.4943	reggae	0.3632	0.1311	0.0193
4823	Dust on the Dial	2	country	0.5153	rock	0.3570	0.1583	0.0128
4819	Glass Meridian	2	classical	0.5562	pop	0.3888	0.1674	0.0233
4829	Tapeworm Blues	4	blues	0.5363	rock	0.3526	0.1837	0.0036
4817	Low Tide Signal	1	jazz	0.5438	hiphop	0.3426	0.2012	0.0243
4828	Afterimage Club	5	disco	0.5516	pop	0.3023	0.2493	0.0246
4825	Marigold Dub	1	reggae	0.4964	hiphop	0.2332	0.2632	0.0273
4821	Cathedral Static	2	classical	0.5556	metal	0.2865	0.2691	0.0288

track_id	title	model_id	top_genre	top_probability	runner_up_genre	runner_up_probability	margin	segment_variance
4817	Low Tide Signal	5	jazz	0.5800	hiphop	0.2843	0.2957	0.0051
4817	Low Tide Signal	4	jazz	0.6002	hiphop	0.2859	0.3143	0.0289
4817	Low Tide Signal	2	jazz	0.6131	hiphop	0.2898	0.3233	0.0058
4823	Dust on the Dial	4	country	0.6131	rock	0.2525	0.3606	0.0179
4825	Marigold Dub	4	reggae	0.5743	hiphop	0.1940	0.3803	0.0246
4827	Stahlwerk	1	metal	0.6295	rock	0.2481	0.3814	0.0117
4822	Hymn for a Server Room	5	classical	0.6804	jazz	0.1499	0.5305	0.0121
4822	Hymn for a Server Room	4	classical	0.7354	jazz	0.1269	0.6085	0.0276
4824	Harbour Lights Waltz	4	country	0.7401	blues	0.1002	0.6399	0.0089

PASS A scoring result with no probability rows has no runner-up, so it is left out rather than shown half empty.

Why: rule I2 says every result has its ten probabilities, but the database alone does not stop a result being written first; the INNER JOIN to the derived table drops it.

Input:

```
INSERT into track_score: track_id=4824, model_id=1, genre_id=2, top_probability=0.5000, posterior_entropy=0.5000, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00
```

Expected: model 1 scores 4824 with no genre_probability rows: 4824 is listed only for model 4, as before.

track_id	title	model_id	top_genre	top_probability	runner_up_genre	runner_up_probability	margin	segment_variance
4820	Second Weather	2	pop	0.4357	disco	0.4304	0.0053	0.0057
4828	Afterimage Club	2	pop	0.4461	disco	0.4273	0.0188	0.0054
4827	Stahlwerk	4	metal	0.4514	rock	0.3636	0.0878	0.0072
4820	Second Weather	4	pop	0.4671	disco	0.3764	0.0907	0.0256
4818	Copper Lanterns	4	blues	0.5077	jazz	0.4133	0.0944	0.0109
4818	Copper Lanterns	1	jazz	0.4819	blues	0.3774	0.1045	0.0111
4826	Plaza at Noon	4	disco	0.4338	reggae	0.3212	0.1126	0.0180
4819	Glass Meridian	1	classical	0.4890	pop	0.3745	0.1145	0.0105
4829	Tapeworm Blues	2	blues	0.4947	rock	0.3777	0.1170	0.0265
4821	Cathedral Static	1	metal	0.4799	classical	0.3612	0.1187	0.0081
4826	Plaza at Noon	5	disco	0.4943	reggae	0.3632	0.1311	0.0193
4823	Dust on the Dial	2	country	0.5153	rock	0.3570	0.1583	0.0128
4819	Glass Meridian	2	classical	0.5562	pop	0.3888	0.1674	0.0233
4829	Tapeworm Blues	4	blues	0.5363	rock	0.3526	0.1837	0.0036
4817	Low Tide Signal	1	jazz	0.5438	hiphop	0.3426	0.2012	0.0243
4828	Afterimage Club	5	disco	0.5516	pop	0.3023	0.2493	0.0246
4825	Marigold Dub	1	reggae	0.4964	hiphop	0.2332	0.2632	0.0273
4821	Cathedral Static	2	classical	0.5556	metal	0.2865	0.2691	0.0288
4817	Low Tide Signal	5	jazz	0.5800	hiphop	0.2843	0.2957	0.0051
4817	Low Tide Signal	4	jazz	0.6002	hiphop	0.2859	0.3143	0.0289
4817	Low Tide Signal	2	jazz	0.6131	hiphop	0.2898	0.3233	0.0058
4823	Dust on the Dial	4	country	0.6131	rock	0.2525	0.3606	0.0179
4825	Marigold Dub	4	reggae	0.5743	hiphop	0.1940	0.3803	0.0246
4827	Stahlwerk	1	metal	0.6295	rock	0.2481	0.3814	0.0117
4822	Hymn for a Server Room	5	classical	0.6804	jazz	0.1499	0.5305	0.0121
4822	Hymn for a Server Room	4	classical	0.7354	jazz	0.1269	0.6085	0.0276
4824	Harbour Lights Waltz	4	country	0.7401	blues	0.1002	0.6399	0.0089

UC-12 Fill a station with recordings that match its blend and window

Five tables: [M5] in multiple_table.sql (WITH) applies the RA's four rules for a station's contents.

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`station`	`station_id, model_id, minimum_ambiguity, maximum_ambiguity, name`	8 rows	with a join
`station_blend`	`station_id, genre_id, weight`	19 rows	with a join
`track_score`	`track_id, model_id, posterior_entropy, genre_id`	27 rows	with a join
`genre_probability`	`track_id, model_id, genre_id, probability`	270 rows	with a join
`track`	`track_id, title, duration, ingest_status`	15 rows	with a join
`genre`	`genre_id, name`	10 rows	with a join

M5 | station, track_score, track, station_blend, genre | multiple_table.sql

BUSINESS NEED

a station stores a genre blend and an ambiguity window, not a fixed list, and the service "returns recordings that genuinely occupy that region" (RA: Overview; Station). Which recordings belong on a station has to be worked out from the business plan's own rules, so the query applies each of them in turn: 1. only scored recordings can be streamed (RA: Track); 2. a station is evaluated against its one nominated model version, "so that a station's contents do not change meaning when a different version scores the catalogue" (RA: Scoring Model 1:N Station); 3. the recording's posterior entropy must lie inside the station's window: the minimum excludes recordings "that sit squarely inside a single genre", the maximum those "the models cannot place with any confidence" (RA: Minimum and Maximum Ambiguity); 4. its top genre must be one of the station's blend genres (RA: Station Blend). The RA stores the top genre and the entropy on the scoring result "so that stations can filter without reading ten rows per score" (RA: Track Score), so the filter reads track_score, not the ten genre_probability rows. The RA does not rank qualifying recordings, so the query orders them by the station's own preference: the blend weight of their top genre, highest first (RA: Weight, "the share of the station's blend given to this genre"), then the higher posterior entropy, the recording further between genres, which is the service's selling point; track_id breaks ties.

QUESTION

For each station, which recordings qualify under its model, window and blend, in what order, and how long would the station run?

INPUT

The Project 1 sample data, read through these tables and columns: `crossfade.track` (15 rows): `duration, ingest_status, title, track_id`; `crossfade.station` (8 rows): `maximum_ambiguity, minimum_ambiguity, model_id, name, station_id`; `crossfade.track_score` (27 rows): `genre_id, model_id, posterior_entropy, track_id`; `crossfade.genre` (10 rows): `genre_id, name`; `crossfade.station_blend` (19 rows): `genre_id, station_id, weight`. Task 3 requirement met: 3.5 a derived table defined with WITH. One row per station_id, track_id.

```
WITH eligible AS (
    SELECT st.station_id,
           st.name AS station,
           t.track_id,
           t.title,
           t.duration,
           g.name AS top_genre,
```

```
           b.weight AS genre_weight,
           s.posterior_entropy
    FROM crossfade.station AS st
    INNER JOIN crossfade.track_score AS s ON s.model_id = st.model_id
    INNER JOIN crossfade.track AS t ON t.track_id = s.track_id
    INNER JOIN crossfade.station_blend AS b ON b.station_id = st.station_id
                                           AND b.genre_id = s.genre_id
    INNER JOIN crossfade.genre AS g ON g.genre_id = s.genre_id
    WHERE t.ingest_status = 'scored'
      AND s.posterior_entropy BETWEEN st.minimum_ambiguity AND st.maximum_ambiguity
),
station_length AS (
    SELECT station_id,
           COUNT(*) AS recordings,
           ROUND(SUM(duration) / 60, 1) AS run_minutes
    FROM eligible
    GROUP BY station_id
)
SELECT e.station_id,
       e.station,
       e.track_id,
       e.title,
       e.top_genre,
       e.genre_weight,
       e.posterior_entropy,
       sl.recordings AS station_recordings,
       sl.run_minutes AS station_run_minutes
FROM eligible AS e
INNER JOIN station_length AS sl ON sl.station_id = e.station_id
ORDER BY e.station_id,
         e.genre_weight DESC,
         e.posterior_entropy DESC,
         e.track_id;
```

EXPECTED OUTPUT: 20 row(s), in this order

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7710	Late Jazz Hop	4817	Low Tide Signal	jazz	0.60	0.4619	1	3.6
7711	Porch and Pedal	4823	Dust on the Dial	country	0.70	0.5065	4	12.6
7711	Porch and Pedal	4824	Harbour Lights Waltz	country	0.70	0.4276	4	12.6
7711	Porch and Pedal	4829	Tapeworm Blues	blues	0.30	0.4989	4	12.6
7711	Porch and Pedal	4818	Copper Lanterns	blues	0.30	0.4628	4	12.6
7712	Neon Waltz	4828	Afterimage Club	pop	0.50	0.5261	4	16.2
7712	Neon Waltz	4820	Second Weather	pop	0.50	0.5158	4	16.2
7712	Neon Waltz	4821	Cathedral Static	classical	0.20	0.4998	4	16.2
7712	Neon Waltz	4819	Glass Meridian	classical	0.20	0.4114	4	16.2

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7713	Iron Choir	4821	Cathedral Static	metal	0.50	0.5700	3	13.5
7713	Iron Choir	4827	Stahlwerk	metal	0.50	0.4784	3	13.5
7713	Iron Choir	4819	Glass Meridian	classical	0.34	0.5310	3	13.5
7714	Harbour Dub	4825	Marigold Dub	reggae	0.60	0.6107	1	3.9
7715	Glitterball Drift	4826	Plaza at Noon	disco	0.34	0.5503	2	7.0
7715	Glitterball Drift	4828	Afterimage Club	disco	0.34	0.5417	2	7.0
7716	Rust Belt Radio	4829	Tapeworm Blues	blues	0.25	0.4989	4	12.6
7716	Rust Belt Radio	4818	Copper Lanterns	blues	0.25	0.4628	4	12.6
7716	Rust Belt Radio	4823	Dust on the Dial	country	0.20	0.5065	4	12.6
7716	Rust Belt Radio	4824	Harbour Lights Waltz	country	0.20	0.4276	4	12.6
7717	Blue Hour	4829	Tapeworm Blues	blues	1.00	0.5220	1	3.2

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A recording whose entropy equals the window's minimum or maximum is on the station; one just outside is not.

Why: no sample score falls exactly on a window edge, so BETWEEN's inclusive edges are never tested.

Input:

```
INSERT into station: station_id=7798, listener_id=20938, model_id=3, name=Edges, minimum_ambiguity=0.40, maximum_ambiguity=0.60,
created_on=2026-09-01

INSERT into station_blend: station_id=7798, genre_id=2, weight=1.00

INSERT into track_score: track_id=4817, model_id=3, genre_id=2, top_probability=0.5000, posterior_entropy=0.4000, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00

INSERT into track_score: track_id=4818, model_id=3, genre_id=2, top_probability=0.5000, posterior_entropy=0.6000, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00

INSERT into track_score: track_id=4819, model_id=3, genre_id=2, top_probability=0.5000, posterior_entropy=0.3999, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00

INSERT into track_score: track_id=4820, model_id=3, genre_id=2, top_probability=0.5000, posterior_entropy=0.6001, model_divergence=0.0100,
segment_variance=0.0100, scored_on=2026-09-01 12:00:00
```

Expected: station 7798 (model 3, window 0.40 to 0.60, blend country 1.00) with four model-3 country scores: 4818 (0.6000) and 4817 (0.4000) are on it, higher entropy first; 4819 (0.3999) and 4820 (0.6001) are not.

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7710	Late Jazz Hop	4817	Low Tide Signal	jazz	0.60	0.4619	1	3.6
7711	Porch and Pedal	4823	Dust on the Dial	country	0.70	0.5065	4	12.6
7711	Porch and Pedal	4824	Harbour Lights Waltz	country	0.70	0.4276	4	12.6
7711	Porch and Pedal	4829	Tapeworm Blues	blues	0.30	0.4989	4	12.6
7711	Porch and Pedal	4818	Copper Lanterns	blues	0.30	0.4628	4	12.6
7712	Neon Waltz	4828	Afterimage Club	pop	0.50	0.5261	4	16.2
7712	Neon Waltz	4820	Second Weather	pop	0.50	0.5158	4	16.2
7712	Neon Waltz	4821	Cathedral Static	classical	0.20	0.4998	4	16.2
7712	Neon Waltz	4819	Glass Meridian	classical	0.20	0.4114	4	16.2
7713	Iron Choir	4821	Cathedral Static	metal	0.50	0.5700	3	13.5
7713	Iron Choir	4827	Stahlwerk	metal	0.50	0.4784	3	13.5
7713	Iron Choir	4819	Glass Meridian	classical	0.34	0.5310	3	13.5
7714	Harbour Dub	4825	Marigold Dub	reggae	0.60	0.6107	1	3.9
7715	Glitterball Drift	4826	Plaza at Noon	disco	0.34	0.5503	2	7.0
7715	Glitterball Drift	4828	Afterimage Club	disco	0.34	0.5417	2	7.0
7716	Rust Belt Radio	4829	Tapeworm Blues	blues	0.25	0.4989	4	12.6
7716	Rust Belt Radio	4818	Copper Lanterns	blues	0.25	0.4628	4	12.6
7716	Rust Belt Radio	4823	Dust on the Dial	country	0.20	0.5065	4	12.6
7716	Rust Belt Radio	4824	Harbour Lights Waltz	country	0.20	0.4276	4	12.6
7717	Blue Hour	4829	Tapeworm Blues	blues	1.00	0.5220	1	3.2

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7798	Edges	4818	Copper Lanterns	country	1.00	0.6000	2	6.7
7798	Edges	4817	Low Tide Signal	country	1.00	0.4000	2	6.7

PASS A rejected recording stays off every station even if a model scored it.

Why: every scored row in the sample belongs to a scored recording, so the ingest-status rule is never needed.

Input:

```
INSERT into track_score: track_id=4831, model_id=2, genre_id=0, top_probability=0.5000, posterior_entropy=0.5000, model_divergence=0.0100, segment_variance=0.0100, scored_on=2026-09-01 12:00:00
```

Expected: rejected track 4831, scored by model 2 as genre 0 at entropy 0.5000 (inside station 7717's window and blend), is not on 7717, which still holds 1 recording.

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7710	Late Jazz Hop	4817	Low Tide Signal	jazz	0.60	0.4619	1	3.6
7711	Porch and Pedal	4823	Dust on the Dial	country	0.70	0.5065	4	12.6
7711	Porch and Pedal	4824	Harbour Lights Waltz	country	0.70	0.4276	4	12.6
7711	Porch and Pedal	4829	Tapeworm Blues	blues	0.30	0.4989	4	12.6
7711	Porch and Pedal	4818	Copper Lanterns	blues	0.30	0.4628	4	12.6
7712	Neon Waltz	4828	Afterimage Club	pop	0.50	0.5261	4	16.2
7712	Neon Waltz	4820	Second Weather	pop	0.50	0.5158	4	16.2
7712	Neon Waltz	4821	Cathedral Static	classical	0.20	0.4998	4	16.2
7712	Neon Waltz	4819	Glass Meridian	classical	0.20	0.4114	4	16.2
7713	Iron Choir	4821	Cathedral Static	metal	0.50	0.5700	3	13.5
7713	Iron Choir	4827	Stahlwerk	metal	0.50	0.4784	3	13.5
7713	Iron Choir	4819	Glass Meridian	classical	0.34	0.5310	3	13.5
7714	Harbour Dub	4825	Marigold Dub	reggae	0.60	0.6107	1	3.9
7715	Glitterball Drift	4826	Plaza at Noon	disco	0.34	0.5503	2	7.0
7715	Glitterball Drift	4828	Afterimage Club	disco	0.34	0.5417	2	7.0
7716	Rust Belt Radio	4829	Tapeworm Blues	blues	0.25	0.4989	4	12.6
7716	Rust Belt Radio	4818	Copper Lanterns	blues	0.25	0.4628	4	12.6
7716	Rust Belt Radio	4823	Dust on the Dial	country	0.20	0.5065	4	12.6
7716	Rust Belt Radio	4824	Harbour Lights Waltz	country	0.20	0.4276	4	12.6
7717	Blue Hour	4829	Tapeworm Blues	blues	1.00	0.5220	1	3.2

PASS A station holds only recordings whose top genre is in its blend, as placed by its own model version.

Why: a new station with a one-genre blend and the widest window isolates the blend and model rules.

Input:

```
INSERT into station: station_id=7797, listener_id=20938, model_id=4, name=One Genre, minimum_ambiguity=0.00, maximum_ambiguity=1.00, created_on=2026-09-01
```

```
INSERT into station_blend: station_id=7797, genre_id=8, weight=1.00
```

Expected: station 7797 (model 4, window 0 to 1, blend genre 8 only) holds just 4825 (model 4 places it in genre 8; model 1 also does, but model 1 is not this station's model): 1 recording, 4825's duration in minutes.

station_id	station	track_id	title	top_genre	genre_weight	posterior_entropy	station_recordings	station_run_minutes
7710	Late Jazz Hop	4817	Low Tide Signal	jazz	0.60	0.4619	1	3.6
7711	Porch and Pedal	4823	Dust on the Dial	country	0.70	0.5065	4	12.6
7711	Porch and Pedal	4824	Harbour Lights Waltz	country	0.70	0.4276	4	12.6
7711	Porch and Pedal	4829	Tapeworm Blues	blues	0.30	0.4989	4	12.6
7711	Porch and Pedal	4818	Copper Lanterns	blues	0.30	0.4628	4	12.6
7712	Neon Waltz	4828	Afterimage Club	pop	0.50	0.5261	4	16.2
7712	Neon Waltz	4820	Second Weather	pop	0.50	0.5158	4	16.2
7712	Neon Waltz	4821	Cathedral Static	classical	0.20	0.4998	4	16.2
7712	Neon Waltz	4819	Glass Meridian	classical	0.20	0.4114	4	16.2
7713	Iron Choir	4821	Cathedral Static	metal	0.50	0.5700	3	13.5
7713	Iron Choir	4827	Stahlwerk	metal	0.50	0.4784	3	13.5
7713	Iron Choir	4819	Glass Meridian	classical	0.34	0.5310	3	13.5
7714	Harbour Dub	4825	Marigold Dub	reggae	0.60	0.6107	1	3.9
7715	Glitterball Drift	4826	Plaza at Noon	disco	0.34	0.5503	2	7.0
7715	Glitterball Drift	4828	Afterimage Club	disco	0.34	0.5417	2	7.0
7716	Rust Belt Radio	4829	Tapeworm Blues	blues	0.25	0.4989	4	12.6
7716	Rust Belt Radio	4818	Copper Lanterns	blues	0.25	0.4628	4	12.6
7716	Rust Belt Radio	4823	Dust on the Dial	country	0.20	0.5065	4	12.6
7716	Rust Belt Radio	4824	Harbour Lights Waltz	country	0.20	0.4276	4	12.6
7717	Blue Hour	4829	Tapeworm Blues	blues	1.00	0.5220	1	3.2
7797	One Genre	4825	Marigold Dub	reggae	1.00	0.6107	1	3.9

UC-13 Learn listener taste from completions, saves and skips

Lives in the associative table play: [M1] in multiple_table.sql joins it to listener and track. record_play [P1] in routines.sql records each play with completed computed from the recording's length (rule I7).

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
play	listener_id, track_id, played_at, seconds_played, completed, reaction	31 rows	with a join
track	track_id, duration	15 rows	with a join
listener	listener_id, display_name	8 rows	with a join

M1 | play, listener, track | multiple_table.sql

BUSINESS NEED

CROSSFADE learns listener taste from what they actually play: how much of each recording they hear, whether they finish it, and whether they save or skip it (RA: Overview, "learning listener taste"; Play).

QUESTION

For each listener, how many recordings did they start, finish, save and skip, and how much of the audio they started did they hear?

INPUT

The Project 1 sample data, read through these tables and columns: `crossfade.play` (31 rows): `completed, listener_id, reaction, seconds_played, track_id`; `crossfade.listener` (8 rows): `display_name, listener_id`; `crossfade.track` (15 rows): `duration, track_id`. Task 3 requirement met: 3.1 an associative table and the two tables it relates, joined with INNER JOIN. One row per listener_id.

```
SELECT l.listener_id,
       l.display_name,
       COUNT(*) AS plays,
       SUM(CASE WHEN p.completed = 1 THEN 1 ELSE 0 END) AS completed_plays,
       SUM(CASE WHEN p.reaction = 'saved' THEN 1 ELSE 0 END) AS saves,
       SUM(CASE WHEN p.reaction = 'skipped' THEN 1 ELSE 0 END) AS skips,
       ROUND(100 * SUM(p.seconds_played) / SUM(t.duration), 1) AS pct_of_audio_heard
FROM crossfade.play AS p
INNER JOIN crossfade.listener AS l ON l.listener_id = p.listener_id
INNER JOIN crossfade.track AS t ON t.track_id = p.track_id
GROUP BY l.listener_id, l.display_name
ORDER BY pct_of_audio_heard DESC,
         l.listener_id;
```

EXPECTED OUTPUT: 8 row(s), in this order

listener_id	display_name	plays	completed_plays	saves	skips	pct_of_audio_heard
20937	dokafor	1	1	0	0	100.0
20936	wenz	4	3	1	0	97.2
20934	priyam	5	4	2	0	95.9
20935	tbarros	4	3	1	0	89.4

listener_id	display_name	plays	completed_plays	saves	skips	pct_of_audio_heard
20938	hito	4	2	1	1	79.1
20933	klind	4	3	1	1	74.6
20932	amara_o	4	1	0	1	64.9
20931	jrivera	5	2	2	2	45.9

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A listener who has played nothing has no taste profile yet.

Why: every sample listener has plays, so the INNER JOIN never has to drop anyone.

Input:

```
INSERT into listener: listener_id=20990, email=quiet@example.com, display_name=quiet, plan_tier=free, signup_date=2026-09-01
```

Expected: listener 20990 (no plays) is not in the result; the 8 sample listeners are.

listener_id	display_name	plays	completed_plays	saves	skips	pct_of_audio_heard
20937	dokafor	1	1	0	0	100.0
20936	wenz	4	3	1	0	97.2
20934	priyam	5	4	2	0	95.9
20935	tbarros	4	3	1	0	89.4
20938	hito	4	2	1	1	79.1
20933	klind	4	3	1	1	74.6
20932	amara_o	4	1	0	1	64.9
20931	jrivera	5	2	2	2	45.9

PASS An abandoned, skipped play lowers the share of audio heard and counts as a skip, not a completion.

Why: the sample's top listener (20937) heard all of their one recording; a second, half-heard play must pull the share down by duration, not by play count.

Input:

```
INSERT into play: listener_id=20937, track_id=4817, played_at=2026-09-01 10:00:00, station_id=NULL, seconds_played=107, completed=0, reaction=skipped, royalty_amount=0.00350
```

Expected: 20937 has 2 plays, 1 completed, 1 skip, and hears 100 * (189 + 107) / (189 + 214) = 73.4 percent.

listener_id	display_name	plays	completed_plays	saves	skips	pct_of_audio_heard
20936	wenz	4	3	1	0	97.2
20934	priyam	5	4	2	0	95.9
20935	tbarros	4	3	1	0	89.4
20938	hito	4	2	1	1	79.1
20933	klind	4	3	1	1	74.6
20937	dokafor	2	1	0	1	73.4
20932	amara_o	4	1	0	1	64.9
20931	jrivera	5	2	2	2	45.9

UC-14 Keep an exact royalty ledger

play, track and artist together: [M2] in multiple_table.sql totals the ledger per artist. routines.sql writes it: record_play [P1] computes each royalty once, with play_royalty [F1], and stores it as a snapshot that a later rate change never rewrites.

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`play`	`listener_id, track_id, played_at, seconds_played, completed, royalty_amount`	31 rows	with a join
`track`	`track_id, artist_id, duration`	15 rows	with a join
`artist`	`artist_id, payout_rate, stage_name`	8 rows	with a join

M2 | artist, track, play | multiple_table.sql

BUSINESS NEED

CROSSFADE pays every artist a per-stream royalty from an exact ledger: each play keeps the royalty it earned when it was recorded, so a later rate change never rewrites what is owed (RA: Overview; Play: Royalty Amount; rule I7).

QUESTION

In this royalty run, how much does each artist earn from the plays of their recordings, and how many of those plays qualified?

INPUT

The Project 1 sample data, read through these tables and columns: `crossfade.artist` (8 rows): `artist_id, stage_name`; `crossfade.track` (15 rows): `artist_id, track_id`; `crossfade.play` (31 rows): `royalty_amount, seconds_played, track_id`. Task 3 requirement met: 3.2 two or more tables joined with LEFT OUTER JOIN. One row per artist_id.

```
SELECT a.artist_id,
       a.stage_name AS payee,
       COUNT(p.track_id) AS plays,
       SUM(CASE WHEN p.seconds_played >= 30 THEN 1 ELSE 0 END) AS qualifying_plays,
       IFNULL(SUM(p.royalty_amount), 0) AS usd_owed
FROM crossfade.artist AS a
LEFT OUTER JOIN crossfade.track AS t ON t.artist_id = a.artist_id
LEFT OUTER JOIN crossfade.play AS p ON p.track_id = t.track_id
GROUP BY a.artist_id, a.stage_name
ORDER BY usd_owed DESC,
         a.artist_id;
```

EXPECTED OUTPUT: 8 row(s), in this order

artist_id	payee	plays	qualifying_plays	usd_owed
1046	Ochre Parade	6	6	0.02160
1042	Velvet Static	6	5	0.02000
1047	Juno Fleet	6	5	0.01500
1044	Low Orbit Choir	4	3	0.01140
1045	Tin Harbour	5	3	0.00960
1048	Saltmarsh Tapes	3	3	0.00900

artist_id	payee	plays	qualifying_plays	usd_owed
1043	Mara Quell	1	1	0.00350
1049	Kestrel Ave	0	0	0.00000

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS An artist whose recordings nobody has played yet is listed, owed zero.

Why: the sample's one artist with no plays (1049) has no uploads either, so the second LEFT OUTER JOIN (track to play) never has to keep a track with no plays.

Input:

```
INSERT into artist: artist_id=9002, stage_name=Silent Upload, country=US, joined_date=2026-09-01, payout_rate=0.0030, verified=1
```

```
INSERT into track: track_id=9401, artist_id=9002, title=Case Track 9401, duration=200, release_date=2026-09-01,
audio_checksum=0x0000000000000000..., ingest_status=scored
```

Expected: artist 9002 with track 9401 and no plays: 0 plays, 0 qualifying, 0 owed, listed after every earning artist.

artist_id	payee	plays	qualifying_plays	usd_owed
1046	Ochre Parade	6	6	0.02160
1042	Velvet Static	6	5	0.02000
1047	Juno Fleet	6	5	0.01500
1044	Low Orbit Choir	4	3	0.01140
1045	Tin Harbour	5	3	0.00960
1048	Saltmarsh Tapes	3	3	0.00900
1043	Mara Quell	1	1	0.00350
1049	Kestrel Ave	0	0	0.00000
9002	Silent Upload	0	0	0.00000

PASS A play under 30 seconds counts as a play but not as a qualifying play (RA rule I7).

Why: checks the 30-second boundary: one play at 29 seconds, one at exactly 30.

Input:

```
INSERT into play: listener_id=20937, track_id=4829, played_at=2026-09-01 10:00:00, station_id=NULL, seconds_played=29, completed=0, reaction=NULL, royalty_amount=0.00000
```

```
INSERT into play: listener_id=20937, track_id=4829, played_at=2026-09-02 10:00:00, station_id=NULL, seconds_played=30, completed=0, reaction=NULL, royalty_amount=0.00350
```

Expected: artist 1048 (track 4829) gains 2 plays and 1 qualifying play; what it is owed grows by the two recorded royalties (0.00000 and 0.00350).

artist_id	payee	plays	qualifying_plays	usd_owed
1046	Ochre Parade	6	6	0.02160
1042	Velvet Static	6	5	0.02000
1047	Juno Fleet	6	5	0.01500
1048	Saltmarsh Tapes	5	4	0.01250
1044	Low Orbit Choir	4	3	0.01140
1045	Tin Harbour	5	3	0.00960
1043	Mara Quell	1	1	0.00350
1049	Kestrel Ave	0	0	0.00000

UC-15 Reject a duplicate upload

Question: What happens when an artist uploads an audio file that is already in the catalogue?

An error case: [E1], the last statement in single_table.sql, kept disabled inside its own comment so the file runs clean; the tests run it on its own and MySQL must refuse it with 1062. upload_track [P2] in routines.sql is the business answering it: its DECLARE EXIT HANDLER FOR 1062 catches the duplicate and tells the uploader which recording is already in the catalogue, instead of stopping.

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`track`	`track_id, artist_id, title, duration, audio_checksum`	15 rows	alone: full
`artist`	`artist_id`	8 rows	with a join

E1 | crossfade.track | single_table.sql

BUSINESS NEED

a re-upload of the same audio file must never become a second catalogue entry earning a second royalty stream; the RA keeps the audio checksum (SHA-256 of the file) to "detect duplicate uploads" (RA: Track: Audio Checksum).

INPUT: a write the database must refuse (kept DISABLED at the end of the file; the test runs it on its own)

```
INSERT INTO crossfade.track (track_id, artist_id, title, duration, release_date, audio_checksum, ingest_status)
SELECT 9999, artist_id, CONCAT(title, ' (re-upload)'), duration, '2026-09-30', audio_checksum, 'awaiting scoring'
FROM crossfade.track
WHERE track_id = 4817;
```

EXPECTED OUTPUT

MySQL raises error **1062 uq_track_audio_checksum**; crossfade.track keeps its 15 rows. Test run: 1062 Duplicate entry '...' for key 'track.uq_track_audio_checksum'